

Szegedi Tudományegyetem Informatikai Intézet

SZAKDOLGOZAT

Földes Balázs

2026

Szegedi Tudományegyetem Informatikai Intézet

Boosted:

AI-támogatott fitness-ökoszisztéma és proaktív edzői platform

Szakdolgozat

Készítette:

Földes Balázs

programtervező
informatikus szakos
hallgató

Témavezető:

Dr. Bilicki Vilmos

egyetemi adjunktus

Szeged

2026

Feladatkiírás

A fitness- és személyi edzői piacon számos edzésnaplózó alkalmazás érhető el, azonban kevés olyan komplex megoldás létezik, amely valódi, proaktív hidat képez az edző és a kliens között. A feladat egy olyan webalkalmazás készítése **React** frontend és **FastAPI/Python** backend technológiák használatával, ahol az edzők és klienseik strukturált keretek között tudják menedzselni a közös munkát.

A kliensek számára a rendszer lehetőséget biztosít a napi edzések, valamint az életmódbeli adatok (alvásidő, hidratáció, stressz-szint) rögzítésére, ezzel elősegítve a tudatos fejlődést. Az elkötelezettséget egy beépített széria-rendszer (streak) és egy interaktív motivációs eszköz (Boost) támogatja.

Az edzői oldal központi eleme a portfólió-menedzsment, ahol az edző valós időben követheti kliensei állapotát és statisztikáit. A szoftver kiemelt funkciója az **adatvezérelt riasztási rendszer**, amely algoritmusok segítségével automatikusan azonosítja a lemorzsolódási kockázatot vagy a kritikus túlterhelést.

Az alkalmazás innovatív megoldásként integrálja a **Gemini mesterséges intelligenciát**, amely a naplózott adatok alapján egyénre szabott, strukturált elemzéseket és javaslatokat generál mindkét felhasználói csoport számára. A jegyzetelési lehetőség és a prediktív AI-dashboard biztosítja, hogy az edzők gyorsabban és hatékonyabban tudjanak reagálni a klienseik fizikai és mentális állapotára, minimalizálva a sérülésveszélyt és maximalizálva az eredményességet.

Tartalmi összefoglaló

Téma megnevezése:

Boosted: AI-támogatott fitness-ökoszisztéma és proaktív edzői platform

A megadott feladat megfogalmazása:

A feladat egy olyan komplex webalkalmazás létrehozása, amely proaktív hidat képez az edzők és klienseik között. A rendszer célja a strukturált portfólió-menedzsment biztosítása, a kliensek napi életmódbeli és biometrikus adatainak (alvás, stressz, hidratáció, edzés) rögzítése, valamint a mesterséges intelligencia (AI) bevonásával prediktív, személyre szabott elemzések és riasztások generálása az eredményesebb és biztonságosabb közös munka érdekében.

Megoldási mód:

A feladat megoldásához frontend oldalon a React keretrendszert (Next.js alapokon), backend oldalon pedig Python nyelvet és a FastAPI keretrendszert alkalmazom.

Alkalmazott eszközök, módszerek:

A fejlesztés modern integrált fejlesztőkörnyezetben (pl. Visual Studio Code) történik. Kliensoldalon a React (Next.js) és a Tailwind CSS biztosítja a gyors és modern felhasználói felületet, míg a grafikonok megjelenítéséhez a Recharts könyvtárat, a dokumentumok exportálásához pedig a jsPDF-et alkalmazom. A szerveroldali logikát és az API végpontokat a nagy teljesítményű FastAPI szolgáltatja, az adatbázis-kezelésről pedig SQLite gondoskodik.

Elért eredmények:

A webalkalmazást egy modern, szétválasztott felhőarchitektúrában élesítettem: a kliensoldalt (frontend) a **Vercel**, a szerveroldali üzleti logikát (backend) a **Render.com** platformján hosztoltam, míg az adatbázis háttérét a **Neon.tech** által szolgáltatott PostgreSQL rendszer biztosítja. A projekt teljes forráskódja feltöltésre került a GitHub-ra, ahol publikusan elérhető, és ahonnan a felhőszolgáltatások a folyamatos integráció és telepítés (CI/CD) révén automatikusan frissülnek.

Az elkészült webalkalmazás egy magas minőségű, reszponzív platform, amely asztali gépen, tableten és mobilon egyaránt kompromisszumok nélkül használható. A beépített gamifikációs elemek (szériák, „Boost” jutalmazás) sikeresen növelik a kliensek elköteleződését, míg a dinamikus AI-elemzések és az automatizált pdf-alapú edzésterv-generálás jelentősen felgyorsítják és skálázhatóvá teszik a szakértői munkát.

Kulcsszavak:

személyi edzés, mesterséges intelligencia, gamifikáció, React, Next.js, FastAPI, Python

TARTALOMJEGYZÉK

FELADATKIÍRÁS.....	1
TARTALMI ÖSSZEFOGLALÓ	2
BEVEZETÉS	6
1. PIACKUTATÁS.....	8
1.1. TRAINERIZE.....	8
1.2. MYFITNESSPAL.....	9
1.3. TRAININGPEAKS	9
2. FUNKCIONÁLIS SPECIFIKÁCIÓ	10
2.1. VENDÉG (REGISZTRÁLATLAN FELHASZNÁLÓ).....	10
2.2. KLIENS (FELHASZNÁLÓ)	10
2.3. EDZŐ (SZAKÉRTŐ).....	11
3. TERVEZETT MEGJELENÉS	12
3.1. NYITÓOLDAL (LANDING PAGE).....	12
3.2. REGISZTRÁCIÓ ÉS BEJELENTKEZÉS.....	12
3.3. EDZŐI VEZÉRLŐPULT	13
3.4. KLIENS VEZÉRLŐPULT.....	14
3.5. NAPI NAPLÓZÁS (DAILY LOG MODAL).....	15
3.6. KLIENS ADATLAP ÉS ELEMZÉS	15
3.7. AI ASSZISZTENS ELEMZŐ KÖZPONT (AI DASHBOARD)	16
4. FELHASZNÁLT TECHNOLOGIÁK	18
4.1. REACT ÉS NEXT.JS.....	18
4.2. TAILWIND CSS	18
4.3. FASTAPI (PYTHON).....	18
4.4. POSTGRESQL, SQLITE ÉS SQLALCHEMY	19
4.5. GOOGLE GEMINI AI	19
4.6. RECHARTS.....	19
4.7. JSPDF	19
4.8. PYDANTIC	20
4.9. KRIPTOGRÁFIAI MODULOK	20
4.10. UVICORN.....	20
4.11. FELHŐSZOLGÁLTATÁSOK (VERCEL ÉS RENDER).....	20
5. RENDSZERARCHITEKTÚRA	21

5.1. KLIENSOLDAL (FRONTEND)	21
5.2. SZERVEROLDAL (BACKEND)	22
6. BELSŐ FELÉPÍTÉSE	23
6.1. FŐ NÉZETEK (VIEWS)	23
6.2. MODÁLIS ÉS UI KOMPONENSEK (MODALS)	24
6.3. API SZOLGÁLTATÁSOK (BACKEND ENDPOINTS)	25
6.4. ADATMODELLEK (DATABASE INTERFACES / SCHEMAS)	25
6.6. ÚTVONALVÁLASZTÁS ÉS JOGOSULTSÁGKEZELÉS (ROUTING & GUARDS).....	27
7. ADATMODELL.....	29
7.1. FELHASZNÁLÓ (USERS)	29
7.2. NAPI NAPLÓ (DAILYLOGS)	30
7.3. HETI EDZÉSTERV (WEEKLYPLANS).....	30
7.4. MEGHÍVÓ (INVITES).....	31
7.5. ADATCSERE OBJEKTUMOK (PYDANTIC SÉMÁK).....	31
7.7. AI KONTEXTUS-OBJEKTUMOK (AI DTOS).....	32
8. BIZTONSÁG ÉS JOGOSULTSÁGKEZELÉS.....	33
8.1. ADATVÉDELEM ÉS KRIPTOGRÁFIA	33
8.2. SZEREPKÖRALAPÚ HOZZÁFÉRÉS VEZÉRLÉS(RBAC).....	34
8.3. BEMENETI ADATOK VALIDÁCIÓJA (PYDANTIC).....	34
8.4. HÁLÓZATI BIZTONSÁG.....	35
8.5. KLIENSOLDALI VÉDELEM.....	35
9. A RENDSZER MAGASSZINTŰ FOLYAMATAI.....	36
9.1. A NAPI BIOMETRIKUS NAPLÓZÁS FOLYAMATA	36
9.2. AZ AI PREDIKTÍV ELEMZÉS FOLYAMATA.....	37
10. FONTOSABB KÓDRÉSZLETEK.....	39
10.1. NAPI BIOMETRIKUS NAPLÓ RÖGZÍTÉSE	39
10.2. MESTERSÉGES INTELLIGENCIA ELEMZÉS	40
10.3. KAPCSOLAT MEGSZAKÍTÁSA ÉS EDZŐI ÉRTÉKELÉS	40
11. TESZTELÉS.....	42
11.1. EGYSÉGTESZTEK (UNIT TESTING)	42
12. MESTERSÉGES INTELLIGENCIA HASZNÁLATA A FEJLESZTÉS SORÁN... 43	
12.1. ALKALMAZOTT ESZKÖZÖK ÉS FELADATKÖRÖK.....	43

12.2. AZ MI KIMENETÉNEK VALIDÁCIÓJA ÉS KONKRÉT HIBÁI	44
12.3. HOL NEM HASZNÁLTAM MESTERSÉGES INTELLIGENCIÁT?	44
12.4. TANULSÁGOK ÉS JÖVŐKÉP	45
13. TAPASZTALATOK, TOVÁBBFEJLESZTÉSI LEHETŐSÉGEK.....	46
IRODALOMJEGYZÉK.....	49
NYILATKOZAT.....	50
KÖSZÖNETNYILVÁNÍTÁS	51
ELEKTRONIKUS MELLÉKLETEK.....	52

Bevezetés

Az egészségtudatos életmód és a személyi edzés piacán egyre nagyobb az igény az adatalapú, transzparens közös munkára. Jelenleg számos edzésnaplózó alkalmazás érhető el, de kevés olyan átfogó megoldás létezik, amely proaktív hidat képez az edző és a kliens között. Ezt az űrt hivatott betölteni a Boosted webalkalmazás, amely egyetlen közös platformon teszi lehetővé a biometrikus adatok naplózását és a szakértői portfólió strukturált menedzselését.

Fontos szempont volt a webalkalmazás gyors és intuitív használata, hogy mind a szakértők, mind a kliensek a tényleges céljaikra tudjanak fókuszálni. Ezt a letisztult, modern felhasználói felület, a könnyen áttekinthető műszerfalak (dashboards) és a vizuális csúszkákkal támogatott gyors napi naplózás biztosítja. A kliensek percek alatt rögzíthetik az alvásidejüket, stressz-szintjüket, vízfogyasztásukat és edzésadataikat.

A hagyományos edző-kliens kapcsolatban sokszor elvesznek a kritikus információk két személyes találkozó között. A lemorzsolódás vagy a túledzés elkerülése érdekében az alkalmazás egy adatalapú, intelligens riasztási rendszert kapott. Az edzők azonnali, kiemelt értesítést látnak a műszerfalukon, ha egy kliens stressz-szintje kritikussá válik, vagy ha napok óta nem rögzített adatot, így időben, proaktívan be tudnak avatkozni a programba.

A pusztán adatrögzítésen túl a platform innovációja a mesterséges intelligencia integrációjában rejlik. A Boosted Neural Engine folyamatosan elemzi a kliensek naplóit, rejtett mintázatokat és trendeket keresve. Ezek alapján személyre szabott, prediktív elemzéseket és azonnal beépíthető javaslatokat generál a kiegészítés megelőzésére és a fejlődés optimalizálására, tehermentesítve ezzel az edzőt és maximalizálva a kliens eredményességét.

A szakmai útmutatás átadására a webalkalmazás egy interaktív heti edzéstervező modult kínál. A szakértők napra és időszakra lebontva tudják elkészíteni a programokat, amelyeket a kliensek a saját felületükön letisztult formában olvashatnak, a beépített exportáló funkciónak köszönhetően pedig akár PDF formátumban le is tölthetik azokat offline használatra.

A tartós életmódváltás egyik legnagyobb kihívása a motiváció fenntartása. A fiatalabb generációk és a modern applikációk elvárásainak megfelelően a Boosted is beépített gamifikációs elemekkel ösztönzi a felhasználókat. A folyamatos adatkitöltést a rendszer "széria" (streak) számlálóval díjazza, míg az edzők egy egyedi, vizuális "Boost" küldésével tudják azonnal, pozitívan megerősíteni a klienseik kitartását.

A webalkalmazás reszponzivitásán is nagy volt a hangsúly. Cél volt, hogy a kliensek az adataikat bárhol – akár útközben vagy az edzőteremben – kényelmesen rögzíthessék

mobiltelefonról, míg az edzők az összetett statisztikákat asztali gépen vagy tableten is átláthatóan tudják elemezni.

1. Piackutatás

A szakdolgozatom elkészítésének legelején területi áttekintést, piackutatást végeztem inspirálódásként. Az áttekintés során olyan webalkalmazásokat kerestem és tekintettem meg, amelyeknek célja a személyi edzők és klienseik közötti kommunikáció támogatása, a fejlődés nyomon követése, valamint az edzésstervezés és az életmód-naplózás megkönnyítése.

		Trainerize	MyFitnessPal	TrainingPeaks	Boosted
FUNKCIÓK	Kliens portfólió menedzsment	TRUE	FALSE	TRUE	TRUE
	Heti edzésstervező naptár	TRUE	FALSE	TRUE	TRUE
	Életmód naplózás (alvás, stressz, víz)	FALSE	TRUE	FALSE	TRUE
	AI-alapú prediktív elemzés	FALSE	FALSE	FALSE	TRUE
	Gamifikáció (Szériák, Boostok)	FALSE	FALSE	FALSE	TRUE
	Edzői figyelmeztetések (Túlterhelés)	FALSE	FALSE	TRUE	TRUE
	PDF edzéssterv exportálás	FALSE	FALSE	FALSE	TRUE
	Modern, letisztult UI (Glassmorphism)	FALSE	FALSE	FALSE	TRUE
	Zárt, meghívó-alapú csatlakozás	TRUE	FALSE	TRUE	TRUE
	Privát edzői jegyzetek és profilozás	TRUE	FALSE	TRUE	TRUE
	Vizuális adatelemzés (Grafikonok)	TRUE	TRUE	TRUE	TRUE
	AI Chat Asszisztens	FALSE	FALSE	FALSE	TRUE

1.1.ábra: Összefoglaló táblázat az áttekintett webalkalmazásokról.

Az 1.1. ábrán látható táblázat a területi áttekintés során készült összefoglaló táblázat, amely az áttekintett webalkalmazások funkcionalitását foglalja össze. A *TRUE* jelzi, ha a webalkalmazásban megtalálható az adott funkcionalitás, a *FALSE* jelzi, ha nem található meg az adott funkcionalitás.

1.1. Trainerize

A Trainerize az egyik legszélesebb körben használt edzői platform, amely a személyi edzők és fitneszstúdiók számára készült. A szoftverben az edzők heti edzésterveket tudnak készíteni, videós gyakorlatkönyvtárat használhatnak, és nyomon követhetik a kliensek alapvető fizikai paramétereit (például testsúly, testzsírszázalék). A webalkalmazás és a hozzá tartozó mobilapplikáció használatához zárt, meghívó-alapú regisztráció szükséges. Hátránya, hogy a rendszer túlságosan a fizikai edzőmunkára fókuszál, így a holisztikus életmód-naplózás (stressz-szint, alvásminőség) nem hangsúlyos benne. Továbbá nem rendelkezik beépített

mesterséges intelligenciával, amely a begyűjtött adatokból proaktívan előrejelezné a kliens esetleges kiegészét, vagy segítené az edzőt a tervek optimalizálásában.

1.2. MyFitnessPal

A MyFitnessPal egy globálisan népszerű, elsősorban végfelhasználóknak (klienseknek) készült alkalmazás. Fő profilja a táplálkozásnaplózás, a kalóriaszámlálás és az alapvető napi aktivitás (pl. lépésszám, vízfogyasztás) nyomon követése. A hatalmas, közösség által is bővített élelmiszer-adatbázisa miatt egyéni célok elérésére kiválóan alkalmas. A legnagyobb hiányossága azonban, hogy nem támogatja az edző-kliens modellt. Nem rendelkezik portfóliómenedzsment felülettel szakértők számára, így az edzők nem tudnak zárt rendszerben heti edzésterveket kiírni a klienseiknek. A rendszerből hiányzik a modern, kétirányú interakció és a gamifikáció is (például egy edzői "Boost" küldése), ami a motiváció fenntartásához elengedhetetlen lenne. A funkciók nagy része bejelentkezéshez, a részletesebb statisztikák pedig prémium előfizetéshez kötöttek.

1.3. TrainingPeaks

A TrainingPeaks a professzionális sportolók, különösen az állóképességi sportágak (triathlon, kerékpár, futás) képviselőinek és edzőinek a legkedveltebb szoftvere. Rendkívül részletes adatokat, teljesítménygrafikonokat és túlterhelési figyelmeztetéseket biztosít, így az edzők mélyrehatóan tudják elemezni a sportolók állapotát. Lehetőség van privát edzői jegyzetek készítésére is. A rendszer hátránya annak extrém komplexitásából fakad. A felhasználói felülete (UI) rendkívül adatsűrű, sokak számára elavult hatást kelt, és a laikus, átlagos fitnesscélokkal rendelkező kliensek számára kifejezetten ijesztő és nehezen értelmezhető lehet. Ezenfelül hiányoznak belőle a modern LLM (Large Language Model) alapú AI asszisztensek, amelyek a nyers adatokat közérthető, szöveges javaslatokká formálnák.

2. Funkcionális specifikáció

A Boosted webalkalmazás célja, hogy strukturált és adatalapú keretet biztosítson a személyi edzők és klienseik közötti közös munkához. Ennek megfelelően a rendszer funkciói szigorúan a felhasználói jogosultságok (szerepkörök) köré épülnek fel. A rendszerben három jól elkülöníthető szerepkört különböztetünk meg: a regisztrálatlan Vendéget, a hitelesített Klienset, valamint a szakmai irányítást végző Edzőt.

2.1. Vendég (Regisztrálatlan felhasználó)

(Regisztrálatlan felhasználó) A Vendég a rendszer publikus felületeihez fér hozzá. Jogosultságai a platform megismerésére és a hitelesítési folyamatok elindítására korlátozódnak.

- Publikus információs felület elérése: Megtekintheti a webalkalmazás nyitóoldalt (Landing Page), ahol megismerheti a rendszer nyújtotta funkciókat és a platform célját.
- Regisztráció szakértőként: Hozzáfér a publikus regisztrációs űrlaphoz, ahol saját fiókot hozhat létre Edző/Szakértő szerepkörben. Kliensként a publikus felületről nem regisztrálhat.
- Bejelentkezés: Meglévő fiók esetén az e-mail cím és a jelszó megadásával bejelentkezhet a rendszerbe.

2.2. Kliens (Felhasználó)

Kliens a rendszer végfelhasználója, akinek a fő célja a saját fejlődésének nyomon követése és az edzőjével való kommunikáció. Fiókot kizárólag egy meglévő edző által generált, egyedi meghívó linken keresztül hozhat létre.

- Napi biometrikus naplózás: Rögzítheti a napi életmód-adatait (alvásidő, stressz-szint, vízfogyasztás, edzés hossza és intenzitása, lépésszám, testsúly) és hangulatát. A rendszer naponta csak egy naplóbejegyzést engedélyez.
- Statisztikák megtekintése: Saját műszerfalán (Dashboard) grafikonokon követheti nyomon az elmúlt napokban rögzített adatainak trendjeit, illetve listázhatja korábbi naplóit.
- Heti edzésterv olvasása: Hozzáfér az edzője által számára kiírt aktuális és korábbi heti edzéstervekhez (csak olvasási joggal).
- Profil menedzsment: Szerkesztheti saját adatait (magasság, testsúly, célok, éttrend/allergiák) és módosíthatja profilképét.

- Gamifikáció (Széria és Boost): Nyomon követheti a folyamatos naplózásért járó "széria" napjainak számát, valamint megkapja és láttatja az edzőjétől érkező motivációs "Boost" pontokat.
- Kapcsolat megszakítása és értékelés: Bármikor dönthet úgy, hogy megszakítja a kapcsolatot az edzőjével. Ekkor a rendszer lehetőséget ad számára, hogy egy 1-től 5-ig terjedő skálán és szövegesen értékelje a szakértő munkáját, mielőtt a fiókja törlésre kerül.

2.3. Edző (Szakértő)

Az Edző a platform professzionális felhasználója, aki a kliensek irányításáért, motiválásáért és a szakmai programok összeállításáért felel.

- Kliensek meghívása és menedzselése: Egyedi, biztonságos regisztrációs linkeket (tokeneket) generálhat, amelyeket e-mailben vagy manuálisan oszthat meg a jövőbeli kliensekkel. A regisztrált klienseket egy listában látja és kezelheti.
- Adatvezérelt portfólió-kezelés: Saját műszerfalán átlátja a teljes csapata statisztikáit (pl. aktív kliensek száma, csapat átlagos stressz-szintje, legaktívabb kliensek).
- Kliens adatlap és privát jegyzetek: Részletesen elemezheti egy kiválasztott kliens napi naplóját grafikonos formában, és privát, a kliens számára nem látható szakmai jegyzeteket fűzhet a profiljához.
- Edzéstervezés és exportálás: Kliensenként, heti bontásban (hétfői kezdőnappal) hozhat létre és módosíthat edzésterveket, amelyeket egy beépített funkció segítségével formázott PDF dokumentumként is letölthet.
- Gamifikációs motiválás: Az aktív és jól teljesítő klienseknek "Boost" motivációs értesítést küldhet (naponta kliensenként maximum egyszer).
- AI Asszisztens használata: Hozzáfér a Gemini AI által hajtott prediktív műszerfalhoz. Itt automatikus elemzéseket kap a kliensek sérülés- vagy kiégésveszélyéről, cél-előrejelzésekről, és proaktív rendszer-riasztásokat lát, ha egy kliens figyelmet igényel (pl. magas stressz vagy elmaradt naplózás).
- Szakmai profil kezelése: Kialakíthatja saját publikus profilját (bemutakozás, tapasztalat, város), amely tartalmazza a korábbi kliensektől kapott átlagos értékelését is. Szükség esetén törölheti az inaktív klienseket a rendszerből.

3. Tervezett megjelenés

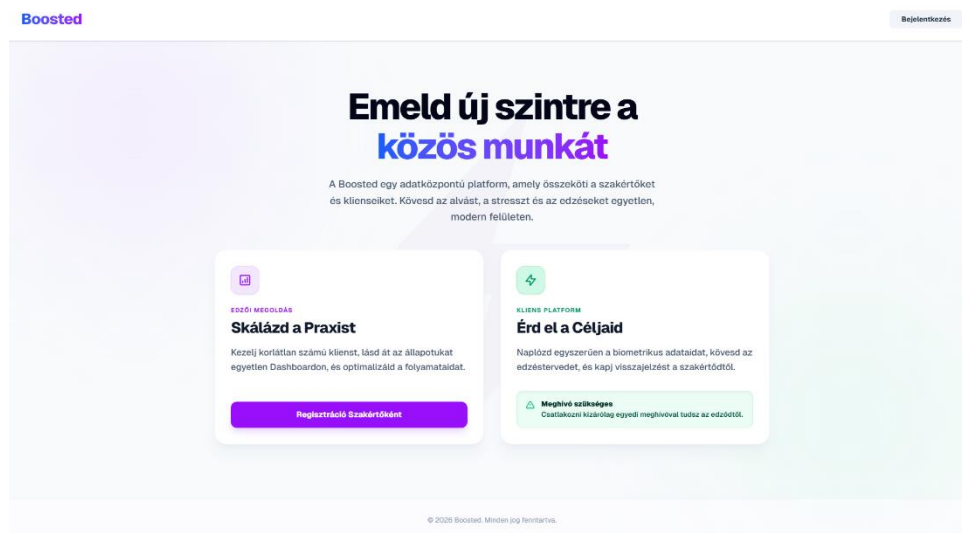
A webalkalmazás vizuális terveit, az úgynevezett wireframe-eket és a magas hűségű (high-fidelity) képernyőterveket a Figma szoftverben készítettem el, még a fejlesztés megkezdése előtt. Az alkalmazás dizájnrendszere a modern "Glassmorphism" és a "Bento Grid" elrendezési stílusokat követi, hangsúlyt fektetve a letisztult, adatvezérelt megjelenésre.

A tervezés során kiemelt prioritás volt a reszponzivitás és a szerepköralapú dinamikus megjelenítés. Ennek megfelelően az edzők és a kliensek felületei bár funkciójukban eltérnek, vizuálisan mégis egy koherens, modern ökoszisztémát alkotnak. A mobil és asztali nézetek közötti folytonosságot a Tailwind CSS biztosítja.

A rendszer színpalettája a szerepkörökhöz igazodik: a kliens nézet domináns színei a nyugtató, egészséget sugárzó smaragdzöld és narancs (emerald & orange), míg a professzionális edzői felület a megbízhatóságot és fókuszt jelképező mély lila és kék (purple & blue) árnyalatokat használja.

3.1. Nyitóoldal (Landing Page)

A nem regisztrált felhasználókat egy modern, animált "hero" szekció fogadja. Az oldal vizuálisan kettéágazik, egyértelműen kommunikálva a platform előnyeit mind az edzők, mind a kliensek számára.

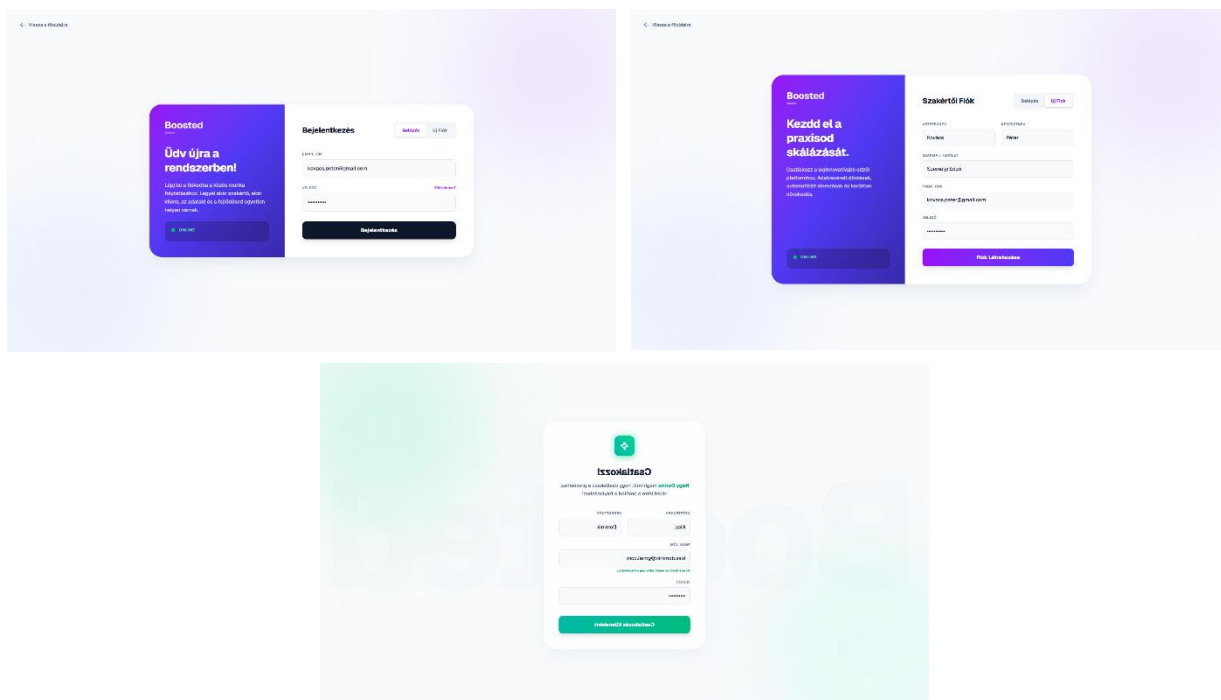


3.1.ábra: A webalkalmazás publikus nyitóoldala.

3.2. Regisztráció és Bejelentkezés

Az alkalmazás belépési felülete egy dinamikus "Split-panel" kialakítást kapott. A bal oldali panelen a márkáépítő (branding) elemek és az aktuális szerepkörnek megfelelő üzenetek

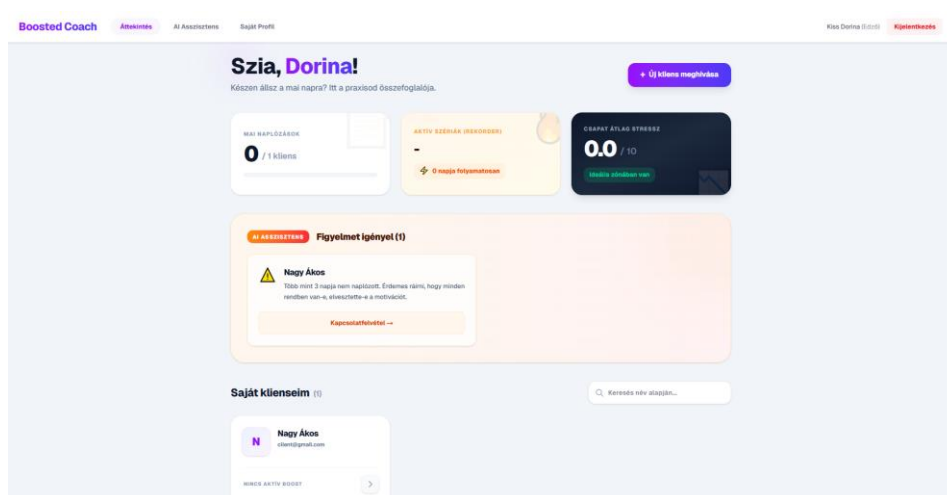
jelennek meg, míg a jobb oldali panelen maga az űrlap kapott helyet. A "Belépés" és az "Új Fiók" állapotok között az oldal zökkenőmentesen, oldalújrátöltés nélkül vált.



3.2.ábra: A bejelentkezési/regisztrációs felület „Split-panel” elrendezése

3.3. Edzői vezérlőpult

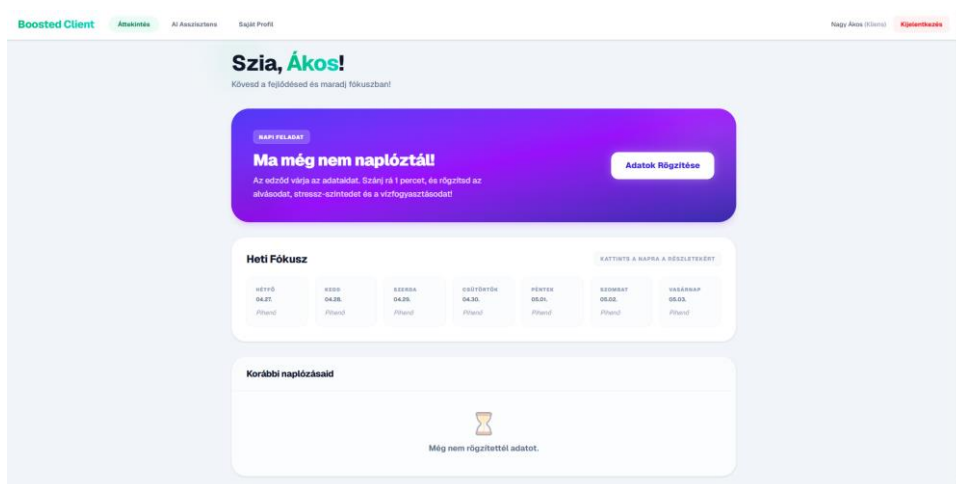
Az edzők számára a központi felület egy átfogó, "Bento Grid" stílusú statisztikai műszerfal. Itt egy pillantással áttekinthetik az aktív szériákat, a mai naplózásokat és a csapat átlagos stressz-szintjét. A felület kiemelt része a mesterséges intelligencia által vezérelt "Figyelmet igényel" szekció, ahol a rendszer riasztásokat jelenít meg a kockázatos állapotú kliensekről. Alatta a kereshető és szűrhető klienslista található.



3.3.ábra: Az edzői vezérlőpult és az AI riasztási modul.

3.4. Kliens vezérlőpult

A kliensek felülete a motivációra és a teendőkre fókuszál. A felső sáv dinamikusan változik: ha a felhasználó még nem naplózott aznap, egy figyelemfelkeltő banner buzdítja az adatok rögzítésére. A felület központi eleme a "Heti Fókusz" naptár, amely vizuálisan, napokra bontva jeleníti meg az edző által kiírt programokat. A korábbi naplóbejegyzések egy külön listában tekinthetők meg.



3.4.ábra: A kliens vezérlőpult, Heti Fókusz naptárral.

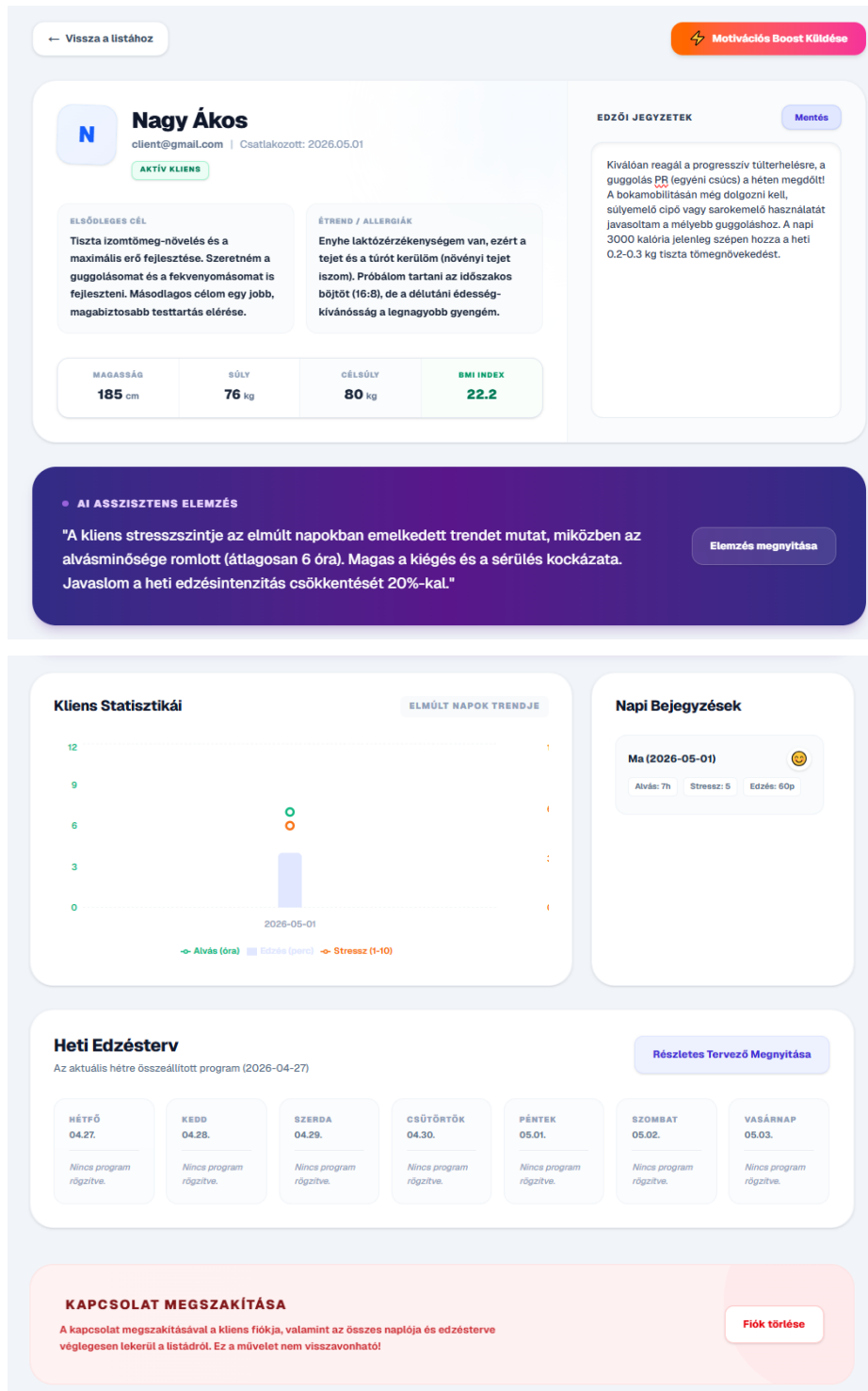
3.5. Napi naplózás (Daily Log Modal)

A napi biometrikus adatok megadása nem egy új oldalon, hanem egy fókuszált modális ablakban (felugró rétegben) történik. A kitöltés felgyorsítása érdekében hagyományos beviteli mezők (input) helyett nagyméretű, mobilon is könnyen kezelhető csúszkákat (rang slider) és ikon alapú választógombokat (pl. hangulat megadásához) alkalmaz a rendszer.

3.5.ábra: Vizuális csúszkákkal támogatott napi naplózás.

3.6. Kliens Adatlap és Elemzés

Amikor egy edző kiválaszt egy klienst, egy részletes analitikai felületre érkezik. Itt a Recharts könyvtár segítségével rajzolt grafikonokon (vonal- és oszlopdiagramok kombinálása) elemezheti a kliens elmúlt napokban mutatott stressz-, alvás- és edzés-trendjeit. Ezen a felületen kezelheti a heti edzéstervet (idősávos tervezővel), és egy külön "Boost" gombbal azonnali motivációs pontot küldhet az ügyfélnek.

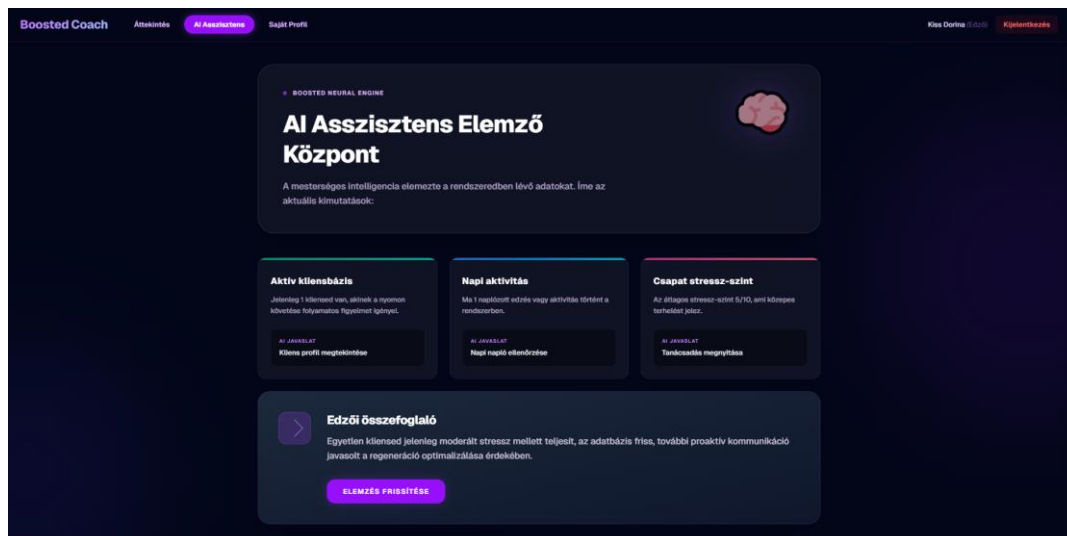


3.6.ábra: Kliens analitikai felület grafikonokkal és edzéstervezővel.

3.7. AI Asszisztens Elemző Központ (AI Dashboard)

Ez a felület mind az edzők, mind a kliensek számára elérhető (eltérő kontextussal). A megjelenés egy futurisztikus, "Neural Engine" stílusú sötét téma, amely kiemeli a funkció különlegességét. Az AI által generált prediktív elemzések (cél előrejelzés, kockázatértékelés,

rejtett mintázatok felismerése) jól áttekinthető kártyákon jelennek meg, amelyek azonnali szakmai "AI javaslatokat" is tartalmaznak.



3.7.ábra: A Gemini AI által generált prediktív elemző műszerfal.

4. Felhasznált technológiák

A Boosted webalkalmazás fejlesztése során kiemelt szempont volt a modern, skálázható és nagy teljesítményű technológiák alkalmazása. A rendszer architektúrája élesen különválasztja a felhasználói felületet (frontend) és a szerveroldali logikát (backend), amelyeket REST API végpontok kötnek össze.

4.1. React és Next.js

A React egy nyílt forráskódú, komponensalapú JavaScript programkönyvtár, amelyet felhasználói felületek (UI) építésére terveztek. A projekt a Reactra épülő Next.js keretrendszert használja (amelyre a kódokban található `use client` direktívák is utalnak). Segítségével az alkalmazás moduláris, újra felhasználható komponensekből épül fel, ami biztosítja a gyors, oldalfrissítés nélküli navigációt (Single Page Application élmény) és a gördülékeny állapotterkezelést (State Management).

4.2. Tailwind CSS

A Tailwind egy „utility-first” CSS keretrendszer, amely előre definiált, alacsony szintű stílusosztályokat biztosít a fejlesztők számára. A hagyományos CSS írása helyett ezeknek az osztályoknak a kombinálásával a HTML (JSX) kódon belül alakítható ki a dizájn. A Boosted reszponzív felülete, a modern Bento Grid elrendezés, valamint a prémium hatást keltő áttetsző felületek (Glassmorphism) és színátmenetek mind a Tailwind CSS segítségével készültek.

4.3. FastAPI (Python)

A szerveroldali (backend) logika megvalósításához a Python nyelven írt FastAPI keretrendszert alkalmaztam. A FastAPI az egyik leggyorsabb modern webes keretrendszer, amely aszinkron működést tesz lehetővé, és automatikus, interaktív API dokumentációt (Swagger UI) generál. Ez a réteg felel a felhasználói kérések (pl. bejelentkezés, adatok lekérése, mentése) villámgyors és biztonságos kiszolgálásáért.

4.4. PostgreSQL, SQLite és SQLAlchemy

Az alkalmazás adatbázis-kezelését fejlesztési (lokális) környezetben a pehelysúlyú SQLite, míg éles (production) felhőkörnyezetben a **Neon.tech** által szolgáltatott **PostgreSQL** relációs adatbázismotor látja el. Az adatbázis és a Python kód közötti kommunikációt az SQLAlchemy nevű ORM (Object-Relational Mapping) könyvtár biztosítja, kiegészítve a psycopg2-binary csomaggal a PostgreSQL kapcsolathoz. Az ORM használatának legnagyobb előnye, hogy a kód hordozható maradt: a Python objektumokként kezelt adatbázis-táblákat a rendszer automatikusan és hibátlanul fordítja le a háttérben az éppen aktuális (SQLite vagy PostgreSQL) SQL dialektusra.

4.5. Google Gemini AI

A rendszer egyik legfőbb innovációja a Google mesterséges intelligencia szolgáltatásának, a Gemini AI-nak (konkrétan a legmodernebb, **Gemini 3.1 Flash Lite** modellnek) az integrálása. A backend a Gemini API segítségével dolgozza fel a kliensek nyers biometrikus és életmód adatait. Az AI ebből prediktív kockázatelemzéseket (sérülés, kiégés), strukturált adathalmazokat (JSON) és szöveges, interaktív asszisztensi válaszokat generál az edzők és a kliensek számára.

4.6. Recharts

A Recharts egy React-alapú, kompozíciós elven működő diagramkészítő könyvtár. A webalkalmazásban az adatvizualizációért felel: ennek a segítségével jelennek meg az edzői és kliens műszerfalakon (dashboardokon) a letisztult, interaktív oszlop- és vonaldiagramok, amelyek az alvást, stresszt és edzésidőt ábrázolják az elmúlt napok tükrében.

4.7. jsPDF

A jsPDF egy nyílt forráskódú kliensoldali JavaScript könyvtár, amely lehetővé teszi PDF dokumentumok dinamikus generálását közvetlenül a böngészőben. A Boosted alkalmazásban az edzői modul használja: a rendszer ennek segítségével alakítja át a JSON formátumban tárolt heti edzésterveket formázott, olvasható, offline is használható PDF fájlakká.

4.8. Pydantic

A FastAPI backend elengedhetetlen része a Pydantic könyvtár használata. Ez a modul felel a kliens és a szerver közötti adatcsere (request és response) szigorú típusellenőrzéséért és validációjáért. A Pydantic segítségével deklarált adatsémák (pl. regisztrációs űrlap, napi napló adatok) garantálják, hogy az adatbázisba és az AI modellhez csak tiszta, formátumilag helyes adatok kerülhessenek be.

4.9. Kriptográfiai modulok

A felhasználói adatok biztonsága kritikus szempont. A rendszer nem tárolja egyszerű szöveggént a jelszavakat; helyette a Python beépített hashlib könyvtárát használja SHA-256 alapú titkosításra (hashing). Emellett a zártkörű, meghívó-alapú regisztrációs rendszerhez a secrets könyvtár generál kriptográfiailag biztonságos, véletlenszerű URL tokeneket, megakadályozva az illetéktelen csatlakozásokat.

4.10. Uvicorn

A Python alapú FastAPI keretrendszer futtatásához egy aszinkron szerverátjáróra van szükség. Ezt a szerepet az Uvicorn tölti be, amely egy villámgyors ASGI (Asynchronous Server Gateway Interface) szerver. Ennek köszönhetően a backend képes egyszerre több kliens kérését (pl. AI generálás, adatbázis-műveletek) blokkolásmentesen, nagy teljesítménnyel kiszolgálni.

4.11. Felhőszolgáltatások (Vercel és Render)

A webalkalmazás élesítése szétválasztott architektúrában (Decoupled Architecture) történt. A React (Next.js) alapú kliensoldal kiszolgálását a **Vercel** platformja végzi, amely globális CDN (Content Delivery Network) hálózaton keresztül biztosítja a gyors betöltődést. A FastAPI backend futtatásáról a **Render.com** gondoskodik. Mindkét szolgáltatás közvetlen összeköttetésben áll a GitHub repozitóriummal, így az új fejlesztések és kódmódosítások (commitok) automatikusan, emberi beavatkozás nélkül települnek az éles szerverekre (CI/CD).

5. Rendszerarchitektúra

A webalkalmazás tervezése során egy modern, szétválasztott (decoupled) kliens-szerver architektúrát alkalmaztam. A rendszer logikailag és fizikailag is két fő komponensből áll: a felhasználói felületért felelős kliensoldalból (Frontend) és az üzleti logikát, valamint az adatkezelést végző szerveroldalból (Backend).

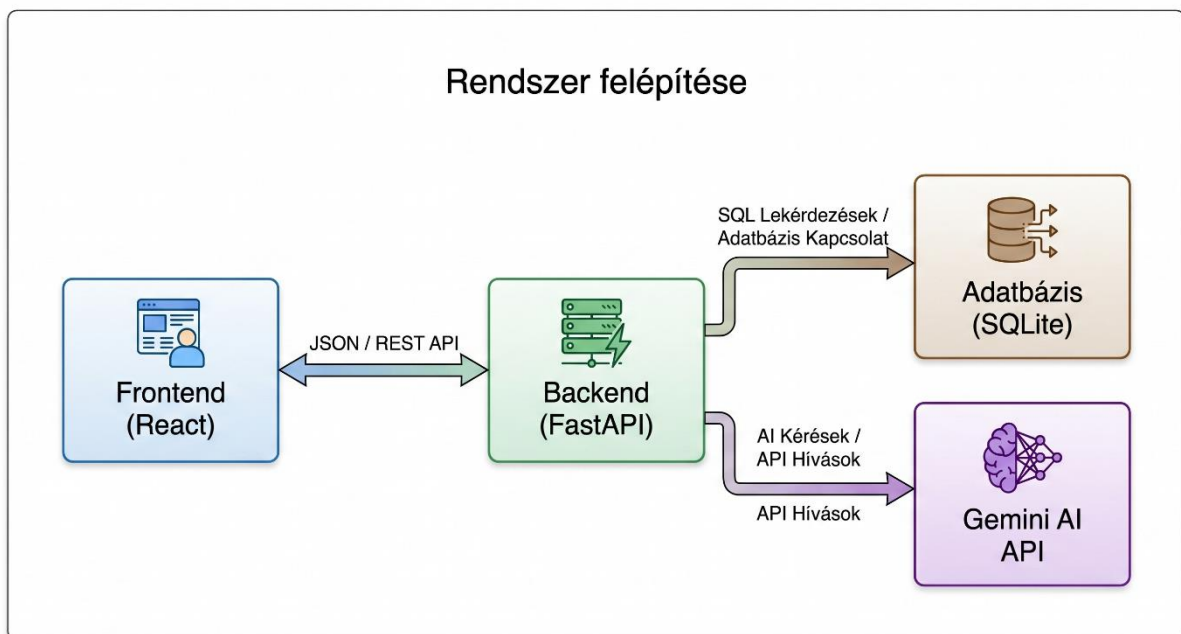
A két réteg közötti adatcsere a REST (Representational State Transfer) architekturális stílus elveit követve, aszinkron HTTP kéréseken keresztül valósul meg. Amikor a kliensoldalnak új információra van szüksége, vagy egy felhasználói interakció (például naplózás, profilmódosítás) történik, a Frontend megfelelő HTTP metódusok (GET, POST, PUT, DELETE) használatával JSON formátumú adatsomagokat küld a Backend felé. A szerveroldal feldolgozza a kérést, elvégzi a szükséges adatbázis-műveleteket vagy AI-generálást, majd a választ szintén strukturált JSON formátumban küldi vissza a prezentációs rétegnek.

5.1. Kliensoldal (Frontend)

A prezentációs réteg, amellyel a felhasználó – legyen az szakértő vagy kliens – közvetlen interakcióba lép. Ez a réteg felelős a vizuális megjelenítésért, az űrlapok kezeléséért, a kliensoldali validációkért és a dinamikus állapotkezelésért (State Management). A hagyományos, különálló HTML és SCSS fájlokkal operáló megközelítés helyett a rendszer a React JSX (JavaScript XML) szintaxisát használja, ahol a struktúra és a logika egyetlen moduláris komponensben egyesül. Az elemek stílusát és reszponzív viselkedését a Tailwind CSS keretrendszer utility-osztályai határozzák meg, amelyek közvetlenül a komponensek szintjén kerülnek definiálásra. Az alkalmazás működéséért és a dinamikus UI-váltásokért (például a modális ablakok megjelenítéséért vagy a diagramok újrarajzolásáért) a React Hook-jai (mint a useState és useEffect) felelnek.

5.2. Szerveroldal (Backend)

A rendszer "agya", amely az üzleti logikáért, a biztonságért és az adattárolásért felel. A FastAPI alapú Backend közvetlenül nem jelenít meg vizuális elemeket a felhasználó számára; kizárólag API (Application Programming Interface) végpontokat szolgáltat a Frontend számára. Ez a réteg felel a felhasználók hitelesítéséért (jelszavak hashelése, tokenek kezelése), az adatbázis-műveletekért (CRUD operációk), valamint az adatok séma-szintű validációjáért (Pydantic modellek). A Boosted alkalmazás szerveroldalának egyik legfőbb egyedi komponense a harmadik féltől származó AI-motorral (Google Gemini) való aszinkron kommunikáció. A Backend összegyűjti és anonimizálja az adatbázisból kinyert biometrikus adatokat, majd kontextusként átadja azokat a mesterséges intelligenciának, hogy a visszaérkező prediktív elemzéseket már feldolgozott formában juttassa el a kliensoldalra.



5.1.ábra: A webalkalmazás rendszerarchitektúrája és a komponensek közötti adatáramlás.

6. Belső felépítése

A webalkalmazás React (Next.js) alapokon nyugszik, amelynek legnagyobb előnye a komponens-alapú, deklaratív architektúra. Ennek köszönhetően a felhasználói felület logikailag jól elkülönülő, állapotvezérelt (state-driven) egységekből épül fel. Az alkalmazás nem használ hagyományos oldalújrátöltést; a navigációt és a különböző felületek megjelenítését a React állapotkezelő Hook-jai (pl. `useState`, `useEffect`) dinamikusan vezérlik.

A webalkalmazás logikai komponenseit két nagy csoportba lehet sorolni. Az egyik csoport a fő **Nézetek (Views)**, amelyek az egész képernyőt kitöltő, alapvető oldalakként funkcionálnak. A másik csoportba a **Modális és UI komponensek (Modals & UI Components)** tartoznak, amelyek a nézetek felett, felugró ablakokként jelennek meg, lehetővé téve a gyors interakciókat anélkül, hogy a felhasználó elveszítené az aktuális kontextust.

6.1. Fő nézetek (Views)

Ezek a komponensek határozzák meg a képernyő fő struktúráját attól függően, hogy a felhasználó éppen az alkalmazás melyik fázisában tart. A navigációt a view állapotváltozó szabályozza.

- **LandingView (Kezdőoldal):** A webalkalmazás nyilvános bemutatkozó felülete. Ez a komponens jelenik meg az alkalmazás megnyitásakor. Modern, úgynevezett "Bento Grid" elrendezéssel, reszponzív kártyákon keresztül mutatja be a szoftver előnyeit elkülönítve az edzők és a kliensek számára.
- **AuthView (Bejelentkezés és Regisztráció):** Ez a felület felel a felhasználók hitelesítéséért és a fióklétrehozásért. Különlegessége, hogy egyetlen dinamikus "Split-panel" elrendezést használ, amely vizuálisan zökkenőmentesen vált a belépési és a regisztrációs űrlapok között, interaktív visszajelzéseket adva a validáció során.
- **DashboardView (Vezérlőpult):** A bejelentkezett felhasználók központi felülete, amely a feltételes renderelésnek köszönhetően dinamikusan változik a felhasználó szerepköre (Edző vagy Kliens) alapján. Belső lapfüleket (Tabs) is tartalmaz:
 - *Áttekintés (Overview):* Edzők számára a klienslistát és a rendszerszintű statisztikákat, kliensek számára a napi teendőket és a naplókat jeleníti meg.
 - *AI Asszisztens (AI):* A Gemini API által generált prediktív elemzéseket és figyelmeztetéseket vizualizáló adatközpont.

- *Profil (Profile)*: A felhasználó saját adatainak (fizikai paraméterek, bemutatkozás) szerkesztésére szolgáló felület.
- **PremiumView (PRO Felület)**: Egy dedikált információs oldal, amely a Boosted PRO előfizetési csomag funkcióit (korlátlan AI asszisztens, PDF exportálás) mutatja be prémium vizuális elemekkel.
- **ClientDetailsView (Kliens Adatlap)**: Az edzői vezérlőpulton belül megnyíló, részletes nézet. Recharts grafikonokon ábrázolja a kiválasztott kliens fejlődését, továbbá tartalmazza az edzői privát jegyzetelés és a "Boost" küldés lehetőségét.

6.2. Modális és UI komponensek (Modals)

Ezek az újra felhasználható "Pure" komponensek felugró ablakokként (dialog) jelennek meg a fő nézetek felett, biztosítva a megszakításmentes felhasználói interakciót.

- **AppAlert**: Globális, dinamikus értesítő komponens, amely a backend műveletek eredményéről (siker, hiba) ad vizuális visszajelzést a szerepkörnek megfelelő színekkel.
- **DailyLogModal**: A kliensek napi adatbeviteli felülete vizuális csúszkákkal (alvás, stressz, víz, edzés).
- **WeeklyPlanEditorModal**: Az edzők számára készült heti edzéstervező felület, időszámítás-megadással és egyéni idővonal-kezeléssel.
- **WeeklyPlanViewerModal**: A kliensek számára készült olvashatóság-fókuszú felület az edzéstervek megtekintésére.
- **LogDetailsModal**: A korábbi naplóbejegyzések részletes, kártyás megjelenítéséért felelős komponens.
- **InviteModal**: Kliensmeghívó tokenek generálását és e-mailes kiküldését kezelő ablak.
- **CoachProfileModal**: A kliens számára teszi elérhetővé a hozzá rendelt edző szakmai bemutatkozását és statisztikáit.
- **ProfilePictureModal**: Külső URL (pl. Imgur) alapján történő profilkép-módosító ablak.
- **PdfExportModal**: Lehetővé teszi az edző számára a korábbi tervek kiválasztását, majd a jsPDF könyvtár segítségével a tervek kliensoldali letöltését.
- **BoostNotificationModal**: A gamifikációs (Boost) jutalmak megjelenítésére szolgáló, egyedi animációval ellátott értesítő ablak.

6.3. API szolgáltatások (Backend Endpoints)

Az adatok kezeléséért és a kliens-szerver kommunikációért a FastAPI alapú backend RESTful végpontjai (endpoints) felelnek. Ezek a végpontok a frontend fetch hívásaira reagálnak, JSON formátumban.

- **Auth Service (/api/login, /api/register):** A bejelentkezést, a jelszavak hashelését (SHA-256) és a jogosultságok kezelését végzi.
- **Profile Service (/api/user/{id}/profile):** A felhasználói profiladatok (magasság, súly, célok, bemutatkozás) lekéréséért és frissítéséért felel.
- **Invite Service (/api/invite):** Kriptográfiailag biztonságos, egyedi tokeneket generál az új kliensek csatlakoztatásához.
- **Client Management (/api/coach/{id}/clients):** Visszaadja az edzőhöz tartozó kliensek listáját és azok legfrissebb heti tervét.
- **Daily Log Service (/api/client/log):** A napi biometrikus naplók adatbázisba írásáért és lekéréséért felelős végpontok.
- **Plan Service (/api/client/{id}/plan):** A heti edzéstervet (JSON stringként tárolt) mentését és módosítását végzi.
- **Boost Service (/api/client/{id}/boost):** A gamifikációs motivációs rendszer logikáját kezeli, növelve a kliens jutalompontjait.
- **AI Dashboard Service (/api/generate-ai-dashboard):** A Google Gemini API-val kommunikáló végpont, amely a rendszerszintű statisztikák alapján átfogó, JSON formátumú prediktív elemzést generál.
- **AI Client Analysis (/api/generate-client-analysis):** Kifejezetten az egyéni kliensek utolsó naplóiból és az edzői jegyzetekből készít részletes kockázat- és célelemzést a Gemini modell segítségével.

6.4. Adatmodellek (Database Interfaces / Schemas)

A backendben az adatok szerkezetét és validációját az SQLAlchemy ORM modellek, valamint a Pydantic adatcsere-objektumok (Schemas) határozzák meg. Ezek garantálják az adatbázis (SQLite) integritását.

- **User Modell:** A felhasználók (edzők és kliensek) közös táblája. Tartalmazza a hitelesítési adatokat (email, password_hash), a fizikai paramétereket (height, weight), a szerepkört (role), valamint a gamifikációs oszlopokat (total_boosts).
- **DailyLog Modell:** A napi naplókat (alvás, stressz, víz, edzésidő, hangulat, lépések) tároló, dátum alapú relációs tábla, amely idegen kulccsal (client_id) kapcsolódik a felhasználóhoz.
- **WeeklyPlan Modell:** A heti edzésterveket tároló tábla, amely JSON stringként perzisztálja az adott hét napjainak feladatait.
- **Invite Modell:** A kiküldött kliensmeghívókat (token, lejáratí idő, státusz) kezelő tábla.
- **Pydantic Sémák (pl. DailyLogCreate, ProfileUpdate):** A kliens felől érkező HTTP kérések (Request Body) adatainak típusellenőrzéséért és validálásáért felelős struktúrák.

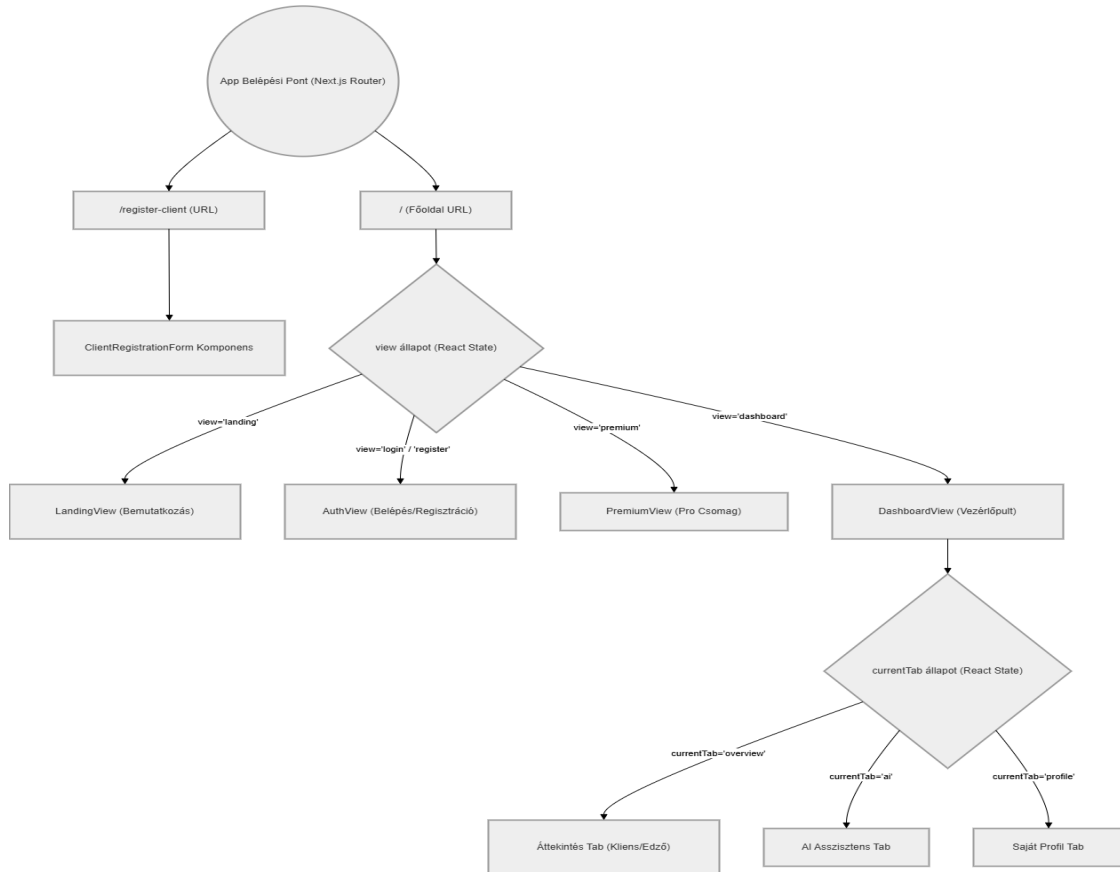
6.5. Adatbetöltési stratégiák (Data Fetching / Resolvers)

Az alkalmazás "Resolverek" helyett a React useEffect hookjait használja az adatok aszinkron betöltésére, még mielőtt a felhasználó megtekintené azokat:

- **Dashboard Data Loader:** A vezérlőpult betöltésekor automatikusan lekéri az edzőhöz tartozó klienseket, a napi statisztikákat, kliens esetén pedig a saját naplókat és edzésterveket.
- **AI Dashboard Loader:** Csak akkor fut le, ha a felhasználó az "AI Asszisztens" fülre navigál. Összegyűjti az aktuális kontextust, elküldi a Gemini AI-nak, majd a visszaérkező JSON adatokból felépíti az intelligens kártyákat.

6.6. Útvonalválasztás és Jogosultságkezelés (Routing & Guards)

A felületek közötti navigáció megoldására a webalkalmazás egy modern, hibrid megközelítést alkalmaz, amely egy belső állapotgépre (State Machine) és a Next.js kiegészítő eszközeire épül.



6.6.ábra: A webalkalmazás navigációs fája és nézethierarchiája.

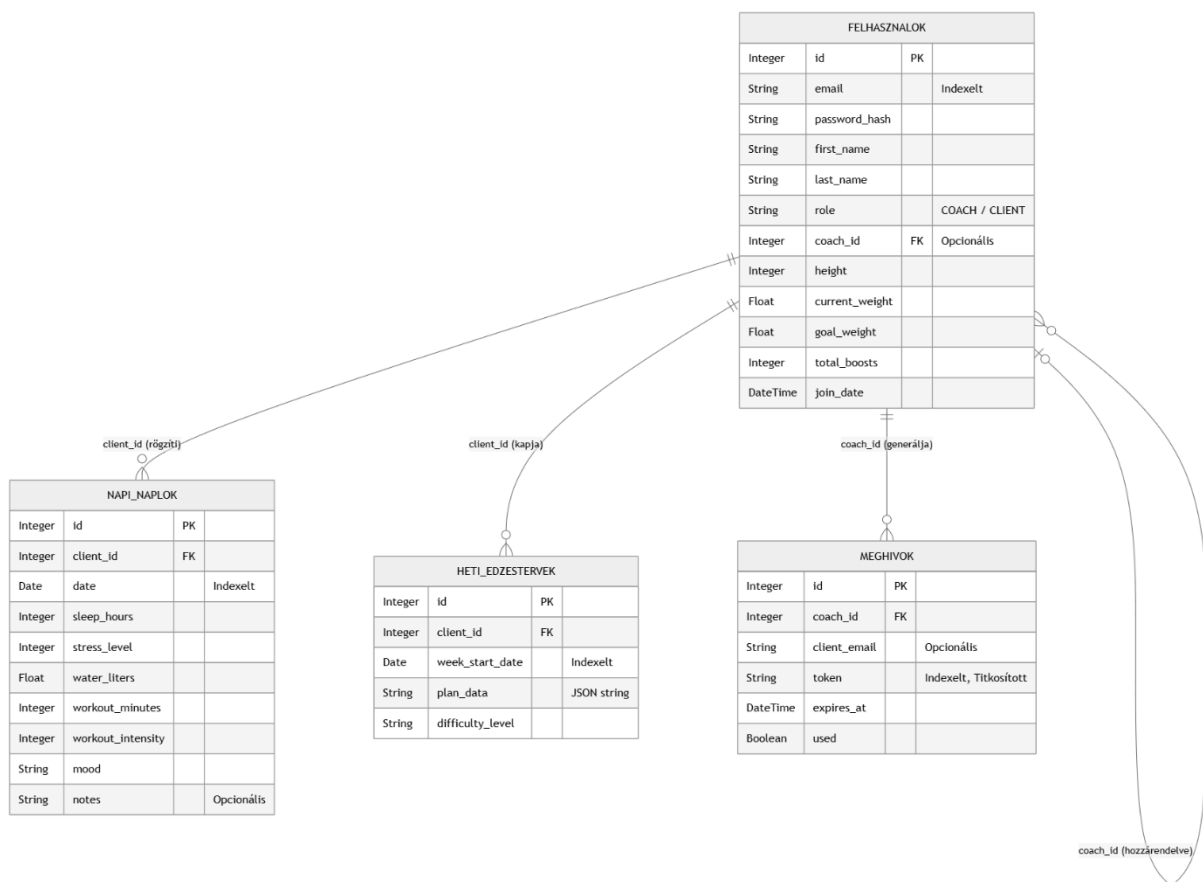
A 6.6. ábrán is látható, hogy a rendszer logikailag több szintre tagolódik. A navigáció és a hozzáférés-vezérlés az alábbi három fő pilléren nyugszik:

- **Állapot-alapú Routing:** Az alkalmazás fő nézetei között a hagyományos oldalújrátöltés helyett a view állapotváltozó (landing, login, dashboard, premium) módosításával történik a váltás. A komponensek dinamikusan cserélődnek le az oldalon. A vezérlőpulton belül egy másodlagos állapot (currentTab) felel az aloldalak közötti lapozásért.
- **Auth Guard (Jogosultságkezelés):** A rendszer szigorúan ellenőrzi a felhasználó bejelentkezési státuszát és a userRole (COACH / CLIENT) állapotot. Az edzők nem férhetnek hozzá a kliens-naplózó felületekhez, a kliensek pedig nem láthatják az edzői statisztikákat és a portfóliókezelőt. Hitelesítés nélkül az alkalmazás a publikus nézeteken (Landing vagy Auth) tartja a felhasználót.

- **URL Paraméter Kezelés (useSearchParams):** Bár a fő alkalmazás Single Page Application-ként működik, a fizikai útvonalak is szerepet kapnak. A regisztrációs oldalon (Next.js Router használatával) az alkalmazás képes kiolvasni a meghívó linkekben szereplő titkosított tokeneket (?token=...), és ez alapján automatikusan azonosítani az új klienshez tartozó edzőt.

7. Adatmodell

A Boosted webalkalmazás adatainak perzisztens tárolásához relációs adatbázis-kezelő rendszert (SQLite) alkalmaztam. Az adatbázis és a szerveroldali üzleti logika (FastAPI) közötti kapcsolatot az SQLAlchemy ORM (Object-Relational Mapping) keretrendszer biztosítja. Szemben a NoSQL (például dokumentumalapú) megoldásokkal, a relációs adatmodellben az adatok szigorúan típusos, normalizált táblákban (táblázatokban) kerülnek eltárolásra. Az egyedek közötti összeköttetéseket idegen kulcsok (Foreign Keys) és azokon alapuló relációk definiálják, ami garantálja a rendszer magas szintű adatintegritását és a konzisztens lekérdezéseket.



7.ábra: A webalkalmazáshoz készült egyed kapcsolat diagramm.

7.1. Felhasználó (Users)

A users táblában a webalkalmazásba regisztrált összes felhasználó adata tárolódik. A rendszer egyetlen, közös táblát használ a szakértők (Edzők) és a Kliensek számára, a kettő közötti megkülönböztetést a role (szerepkör) mező végzi. A kliensek esetében a coach_id mező biztosítja a relációt a hozzájuk rendelt szakértő felé. Egy felhasználóról eltároljuk az azonosításhoz szükséges email címet és a kriptográfiailag titkosított jelszót (SHA-256

password_hash), valamint a vezeték- és keresztnévet. Ezen felül a tábla számos opcionális profiltulajdonságot tartalmaz:

- **Fizikai és cél adatok:** Magasság, jelenlegi súly, célsúly, elsődleges cél, étrend és allergiák.
- **Szakértői identitás adatok:** Város, szakmai bemutatkozás (bio), tapasztalat évei, motivációs jelige, csatlakozás dátuma, és a profilkép URL-je.
- **Gamifikációs adatok:** Eltároljuk a kliens által összegyűjtött motivációs pontokat (total_boosts), valamint egy logikai (boolean) értéket, ami jelzi, ha a kliensnek olvasatlan jutalma van (has_unseen_boost). A felhasználók a saját adataikat szabadon módosíthatják, a jelszavak tárolása pedig egyirányú titkosítással történik.

7.2. Napi napló (DailyLogs)

A daily_logs táblában a kliensek által napi szinten rögzített biometrikus és életmód-adatok találhatók. Ezeket a bejegyzéseket kizárólag a bejelentkezett kliensek hozhatják létre a saját fiókjukban, de a hozzájuk rendelt edzők is jogosultak a lekérdezésekre és elemzésekre. Minden naplóbejegyzés idegen kulccsal (client_id) kapcsolódik a users táblához. Egy bejegyzésből eltároljuk a dátumot, az alvás mennyiségét (órában), a stressz-szintet (1-10-es skálán), a vízfogyasztást (literben), az edzésidőt (percben), az edzésintenzitást, valamint az aznapi hangulatot és egy opcionális szöveges megjegyzést. Extra adatként eltárolható a lépésszám és a testsúly is. A rendszer biztosítja, hogy egy kliens egy adott napra csak egyetlen naplót rögzíthessen, elkerülve a duplikációkat.

7.3. Heti edzésterv (WeeklyPlans)

A weekly_plans tábla a szakértők által kiírt, klienseknek szóló edzésprogramokat perzisztálja. Új tervet csak edzői jogosultsággal rendelkező felhasználó hozhat létre, illetve módosíthat, a kliensek ezeket az adatokat csak olvasási (Read) joggal érhetik el. A tábla a flexibilitás érdekében egy hibrid tárolási megoldást alkalmaz: bár relációs tábláról van szó, maga a napokra lebontott konkrét program egy JSON formátumú stringként (plan_data) kerül mentésre. Ezzel a módszerrel a heti feladatok elrendezése dinamikus maradhat anélkül, hogy minden egyes naphoz külön oszlopot kellene létrehozni. Eltároljuk továbbá a hét kezdődátumát (amely minden esetben hétfőre kell, hogy essen), a terv nehézségi szintjét, és természetesen a tulajdonos kliens azonosítóját (client_id).

7.4. Meghívó (Invites)

Az invites tábla a rendszer zárt, biztonságos regisztrációját felügyeli. Mivel kliensként csak edzői invitálással lehet csatlakozni, ebben a kollekcióban tárolódnak az ideiglenes belépési engedélyek. Eltároljuk a meghívót generáló edző azonosítóját, az opcionálisan megadott kliens email címet, valamint a kriptográfiailag biztonságos, 32 bájtos véletlenszerű tokent. Minden meghívónak van egy lejárat ideje (`expires_at`), amely a generálástól számított 24 óra, továbbá egy logikai mező (`used`) jelzi, ha a tokent már felhasználták egy sikeres regisztráció során, meggátolva a linkek többszörös felhasználását.

7.5. Adatsere objektumok (Pydantic sémák)

Bár az adatbázis fizikai szerkezetét az SQLAlchemy modellek határozzák meg, a rendszer belső felépítésének kritikus részét képezik a Pydantic sémák. Ezek nem fizikai táblák, hanem szigorúan típusos adatsere-interfészek (DTO – Data Transfer Object). Ezek felelnek a frontend felől érkező JSON HTTP kérések (Request Body) validálásáért. Ide tartoznak például a `DailyLogCreate` vagy a `ProfileUpdate` sémák. Szerepük, hogy a nyers adatok még az adatbázis-műveletek és az AI prompt generálás előtt típusellenőrzésen essenek át, garantálva, hogy például a "vízfogyasztás" mezőbe csakis érvényes lebegőpontos szám (Float) kerülhessen.

7.6. Adatbázis-kapcsolat és munkamenet-kezelés (Session Management)

A webalkalmazás hibrid adatbázis-kezelést alkalmaz, amelyet környezeti változók (Environment Variables) vezérelnek, zökkenőmentes átmenetet biztosítva a lokális fejlesztés és a felhőalapú élesítés (deployment) között. A kapcsolat inicializálásánál (engine) a kód dinamikusan ellenőrzi a `DATABASE_URL` változót. Fejlesztési környezetben a rendszer a pehelysúlyú, fájlalapú SQLite (`sqlite:///./boosted.db`) motort használja. Mivel a FastAPI aszinkron és többszálú (multi-threaded) kéréseket szolgál ki, SQLite használata esetén kulcsfontosságú a `check_same_thread=False` paraméter megadása, amely megakadályozza a szálak közötti adatbázis-hozzáférés zárolási (lock) hibáit. Éles környezetben az alkalmazás automatikusan átvált a Neon.tech által szolgáltatott robusztus PostgreSQL szerverre, elhagyva a lokális paramétereket.

A kérések (HTTP Requests) életciklusa alatt az adatbázis-munkameneteket a FastAPI beépített függőséginjektálási (Dependency Injection) rendszere kezeli. A `get_db()` generátor függvény garantálja, hogy minden beérkező API hívás egy friss, izolált adatbázis-munkamenetet (Session) kapjon, majd a kérés lefutása után – a finally blokknak köszönhetően – a kapcsolatot

biztonságosan lezáruljon. Ez az architektúrális döntés kritikus a memóriaszivárgás és a nyitva maradt adatbázis-kapcsolatok (connection leak) elkerülése érdekében.

7.7. AI Kontextus-objektumok (AI DTOs)

A hagyományos adatbázis-sémák mellett a rendszer speciális átmeneti objektumokat is használ a harmadik féltől származó szolgáltatásokkal való kommunikációra. A mesterséges intelligencia (Gemini) integrációjához dedikált Pydantic sémák készültek (AIDashboardRequest, ClientAnalysisRequest, ChatRequest). Ezek a sémák nem kerülnek mentésre a relációs adatbázisban; egyetlen céljuk, hogy a frontendről érkező, nyers promptokat és a bejelentkezett felhasználó szerepkörét (user_type) becsomagolják, validálják, majd kontextusként adják át a Generative AI modell számára.

8. Biztonság és Jogosultságkezelés

Egy modern webalkalmazás esetében a felhasználói adatok védelme és az illetéktelen hozzáférések megakadályozása kritikus fontosságú, különösen egy olyan platformnál, amely érzékeny biometrikus és életmódbeli adatokat kezel. A Boosted alkalmazás egy többrétegű (multi-layered) biztonsági modellt alkalmaz, amely a hálózati kommunikációtól kezdve az adatbázison át egészen a kliensoldali felületig biztosítja a rendszer integritását. A szerveroldali végpontok védelmét és az állapotmentes (stateless) munkamenet-kezelést **JWT (JSON Web Token)** alapú hitelesítés, míg az érzékeny konfigurációs adatok (pl. API kulcsok) védelmét környezeti változókat tartalmazó .env fájlok biztosítják.

8.1. Adatvédelem és Kriptográfia

A felhasználók jelszavait a rendszer soha nem tárolja egyszerű szöveges (plain text) formátumban. Az autentikációs folyamat során a jelszavak titkosításáért az iparági standardnak számító **bcrypt** algoritmus felel, elhagyva az elavult, egyszerű SHA-256 alapú megoldásokat. A bcrypt algoritmus automatikusan egyedi „sót” (salt) generál és ad hozzá minden jelszóhoz a titkosítás (hashing) előtt. Ennek köszönhetően még azonos jelszavak esetén is teljesen eltérő hash lenyomatok keletkeznek, ami garantálja, hogy egy esetleges adatbázis-szivárgás esetén a jelszavak úgynevezett szivárványtáblás (rainbow table) támadásokkal is visszafejthetetlenek maradjanak. Bejelentkezéskor a szerver a beérkező jelszót nem egyszerűen újra hasheli, hanem a bcrypt dedikált ellenőrző módszerével (checkpw) hitelesíti azt az adatbázisban tárolt sós lenyomattal.

A zártkörű regisztrációs folyamat védelmét a Python secrets kriptográfiai modulja biztosítja. Amikor egy edző új klienst hív meg, a rendszer a secrets.token_urlsafe(32) módszerrel egy 32 bájtos, kriptográfiailag biztonságos, véletlenszerű URL tokent generál. Ez megakadályozza, hogy illetéktelenek kitalálható linkek (pl. növekvő azonosítók) megadásával férjenek hozzá a regisztrációs felülethez.

8.2. Szerepköralapú hozzáférés vezérlés(RBAC)

A rendszer a jogosultságokat Szerepköralapú Hozzáférés-vezérlés (Role-Based Access Control) segítségével kezeli. A felhasználókat a role mező alapján két fő csoportra (Edző / Kliens) osztjuk, a be nem jelentkezett felhasználókat pedig Vendégként kezeljük. A Backend API végpontok és a Frontend felületek szigorúan izolálják az adatokat a szerepkörök között.

Jelmagyarázat: C = Létrehozás (Create), R = Olvasás (Read), U = Módosítás (Update), D = Törlés (Delete)

Szerepkör	Fiók / Profil	Napi Napló	Heti edzésterv	Kliens meghívó	Rendszer statisztika
Vendég	C (Regisztráció)	-	-	R (Token validáció)	-
Kliens	RU (Saját)	CR (Saját)	R (Saját)	-	-
Edző	RU (Saját), R(Kliense)	R (Kliensei)	CRU (Kliensei)	C	R (Saját csapat)

8.ábra: Objektumokhoz és erőforrásokhoz való hozzáférés.

8.3. Bemeneti adatok validációja (Pydantic)

A NoSQL adatbázisokba épített "szabályok" helyett a FastAPI a Pydantic könyvtárat használja a bejövő HTTP kérések (Request Body) ellenőrzésére. Ez a köztes réteg garantálja a sémaalapú validációt.

```
class ClientRegister(BaseModel):
    token: str
    email: str
    password: str
    first_name: str
    last_name: str
```

```
class DailyLogCreate(BaseModel):
    client_id: int
    date: str
    sleep_hours: int
    stress_level: int
    water_liters: float
    workout_minutes: int
    mood: str
    notes: str = None
    workout_intensity: int = 0
    steps: int = None
    weight: float = None
```

8.3.ábra: Pydantic adatsémák és típusvalidáció a backend kódban.

A fenti ábrán látható sémák biztosítják, hogy az adatbázisba csak típushelyes (például sleep_hours esetén kizárólag egész szám, email esetén megfelelő formátumú sztring) adat kerülhessen. Ha a kliensoldalról érkező adat hibás, a Backend automatikusan elutasítja a kérést

egy 422-es (Unprocessable Entity) HTTP hibakóddal, megakadályozva az injektálós támadásokat és az adatbázis korrupcióját.

8.4. Hálózati biztonság

A böngészők beépített védelmének kezelésére a rendszer implementálja a FastAPI CORSMiddleware (Cross-Origin Resource Sharing) komponensét. Mivel a webalkalmazás szétválasztott architektúrára épül (a React Frontend és a FastAPI Backend külön porton vagy szerveren fut), ezen szabályok definiálása elengedhetetlen. A middleware szabályozza, hogy mely külső domainek (allow_origins), HTTP metódusok és fejlécek kommunikálhatnak az API végpontokkal, védelmet nyújtva a jogosulatlan külső szkriptekkel szemben.

8.5. Kliensoldali Védelem

A biztonságra a Frontend oldalon (React) is kiemelt figyelem hárul. A felhasználók már az űrlapok kitöltésekor azonnali, interaktív hibaüzeneteket és HTML5 validációt (pl. required, type="email") kapnak, ami javítja a felhasználói élményt és csökkenti a szerver felesleges terhelését.

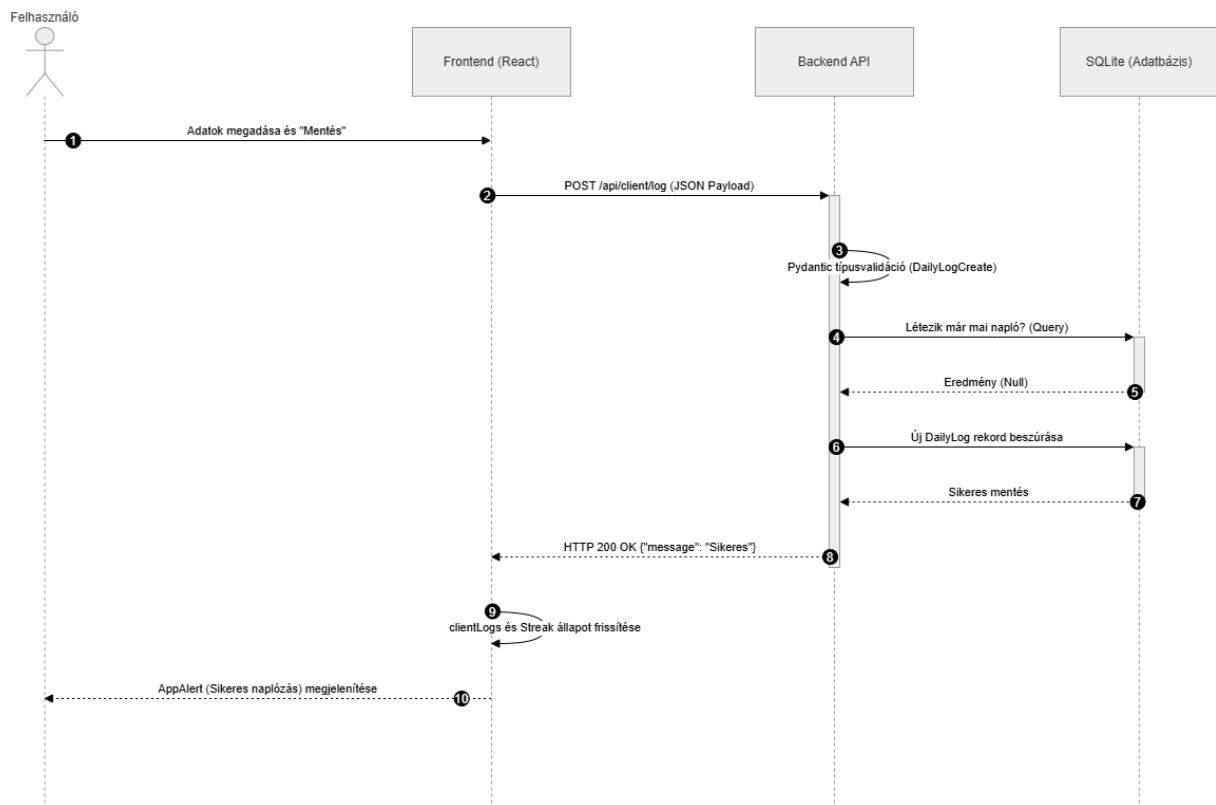
Bizonyos oldalakra (pl. DashboardView) való navigálás védve van az illetéktelen (Vendég) hozzáféréstől. A React komponens állapotváltozói (userRole és userId) működnek "Auth Guard" (Hitelesítési Védvonal) gyanánt: amennyiben a felhasználó nem rendelkezik érvényes munkamenettel (session), a rendszer automatikusan a publikus bemutatkozó vagy a bejelentkezési oldalra irányítja, elrejtve a belső üzleti logikát és a felületeket.

9. A rendszer magasszintű folyamatai

A Boosted webalkalmazás modern, aszinkron kliens-szerver architektúrára épül. Az alkalmazás magasszintű folyamatai során a felhasználói interakciók nem közvetlenül módosítják az adatbázist, hanem egy szigorúan szabályozott adatáramlási (Data Flow) láncon mennek keresztül a frontenden, a backend végpontokon, a validációs rétegeken és az ORM motoron keresztül. Ebben a fejezetben a rendszer két legkritikusabb logikai folyamatát mutatom be.

9.1. A napi biometrikus naplózás folyamata

A napi naplózás a kliensek leggyakoribb interakciója a platformmal. Ennek a folyamatnak garantálnia kell a gyorsaságot, a hibás adatok kiszűrését, valamint a napi duplikációk megakadályozását.



9.1.ábra: A napi naplózás folyamatát bemutató UML szekvenciadiagram.

A 9.1-es ábrán látható a napló rögzítésének szekvenciadiagramja. A folyamat a kliensoldali modális ablak (DailyLogModal) megnyitásával és az adatok (alvás, stressz, víz, stb.) csúszkákon történő megadásával kezdődik. Amikor a felhasználó a "Mentés" gombra kattint, a React komponens handleSaveLog aszinkron függvénye állítja össze a JSON adatcsomagot

(Payload). A rendszer automatikusan hozzárendeli a kéréshez a mai dátumot és a bejelentkezett kliens azonosítóját (`client_id`).

A kérés egy HTTP POST metódussal érkezik be a FastAPI backend `/api/client/log` végpontjára. Mielőtt az üzleti logika elindulna, a beérkező adat a `DailyLogCreate` Pydantic sémán halad keresztül, amely elvégzi a szigorú típusvalidációt. Ha az adatok helyesek, az adatbázis-munkamenet (`SessionLocal`) lekérdezi, hogy az adott felhasználó a mai napon rögzített-e már adatot. Duplikáció esetén a szerver egy 400-as HTTP hibával tér vissza.

Sikeres ellenőrzés után az SQLAlchemy ORM létrehozza az új `DailyLog` objektumot és elmenti az SQLite adatbázisba. A folyamat érdekessége, hogy ha a felhasználó megadta a testsúlyát, az algoritmus nemcsak a naplót hozza létre, hanem automatikusan frissíti a `users` táblában szereplő `current_weight` attribútumot is. Végezetül a szerver sikeres választ küld, a Frontend pedig újratöltés nélkül (State frissítéssel) frissíti a grafikont és a "Széria" számlálót, majd egy zöld `AppAlert` értesítéssel jelez a felhasználónak.

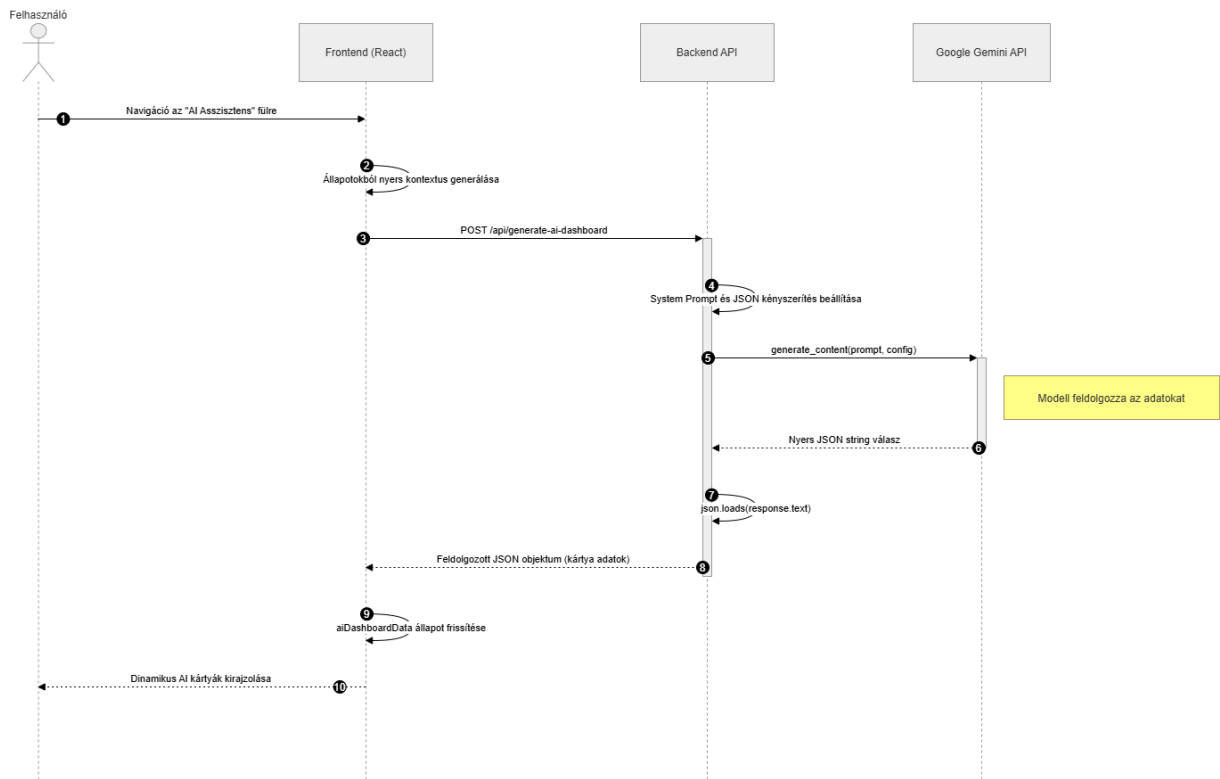
9.2. Az AI prediktív elemzés folyamata.

A rendszer legösszetettebb folyamata a Google Gemini mesterséges intelligencia integrációja, amely a nyers adatbázis-rekordokat vizuálisan értelmezhető, interaktív tanácsokká alakítja át.

Amint a felhasználó az "AI Asszisztens" fülre navigál, a React `useEffect` hookja elindítja a `fetchAiDashboard` folyamatot, és a képernyőn megjelenik egy töltőképernyő. A Frontend az adott felhasználó szerepköre alapján felépíti az AI kontextusát (Context Data): edzők esetén a csapat létszámát, az átlagos stressz-szintet és a riasztásokat, kliens esetén pedig a célokat és az utolsó három napi napló nyers adatait fűzi össze egy nyers szöveggé.

A backend a `/api/generate-ai-dashboard` végponton fogadja a kérést. Itt a FastAPI inicializálja a **gemini-3.1-flash-lite-preview** modellt, majd a System Prompt (rendszerutasítás) segítségével definiálja az AI személyiségét és kényszeríti a kimeneti formátumot. A `generation_config` objektumon keresztül a backend előírja, hogy az AI válasza kizárólag érvényes JSON formátumú lehet (`response_mime_type: "application/json"`).

A Google szervereivel való aszinkron kommunikáció után a kapott nyers JSON sztringet a Python beépített `json.loads` metódusa alakítja vissza szótár (dictionary) objektummá, majd továbbítja a kliensnek. A Frontend a visszaérkező adatokból (pl. `card1_title`, `summary_text`) felépíti a Bento Grid elrendezésű dinamikus kártyákat, elrejtve a töltőképernyőt.



9.2.ábra: Az adatalapú AI Dashboard generálásának szekvenciadiagramja.

10. Fontosabb kódrészletek

Ebben a fejezetben a Boosted webalkalmazás legfontosabb, felhasználói interakciókhoz kötődő funkcióit és azok programkódját mutatom be.

10.1. Napi biometrikus napló rögzítése

A kliensek napi adatainak rögzítése a főoldalon megjelenő DailyLogModal felugró ablakban történik. A felhasználók egy vizuálisan intuitív, csúszkákkal (range inputs) és gombokkal ellátott űrlapon tudják megadni az alvásuk, stressz-szintjük, vízfogyasztásuk és edzésük paramétereit. Az űrlap beküldésekor a `handleSaveLog()` aszinkron függvény hívódik meg (10.1.1. ábra).

```
const handleSaveLog = async () => {
  setIsActionLoading(true);
  try {
    const today = new Date().toISOString().split("T")[0];

    const finalMinutes = didWorkout ? workoutMinutes : 0;
    const finalIntensity = didWorkout ? workoutIntensity : 0;

    const payload = {
      client_id: userId, date: today, sleep_hours: sleepHours, stress_level: stressLevel,
      water_liters: waterLiters, workout_minutes: finalMinutes, workout_intensity: finalIntensity,
      mood: mood, notes: logNotes || ""
    };

    if (steps) payload.steps = parseInt(steps);
    if (dailyWeight) payload.weight = parseFloat(dailyWeight.replace(',', '.'));

    const res = await fetch("https://boosted-backend-bvvv.onrender.com/api/client/log", {
      method: "POST", headers: { "Content-Type": "application/json" }, body: JSON.stringify(payload),
    });

    if (res.ok) {
      setHasLoggedToday(true); setIsLogModalOpen(false);
      const newLogs = [...payload, id: Date.now(), ...clientLogs];
      setClientLogs(newLogs); setCurrentStreak(calculateStreak(newLogs));
      if (dailyWeight) { setClientProfile(prev => ({ ...prev, weight: parseFloat(dailyWeight.replace(',', '.')) })); }

      setSleepHours(7); setStressLevel(5); setWaterLiters(2.0);
      setDidWorkout(true); setWorkoutMinutes(60); setWorkoutIntensity(5);
      setSteps(""); setDailyWeight(""); setMood("😊 Szuper"); setLogNotes("");

      triggerAlert("Napi napló sikeresen elmentve!", "success");
    } else {
      const err = await res.json();
      if (Array.isArray(err.detail)) {
        const errorMessages = err.detail.map(e => `${e.loc[e.loc.length-1]}: ${e.msg}`).join("\n");
        triggerAlert("Validációs hiba:\n" + errorMessages, "error");
      } else {
        triggerAlert(err.detail, "error");
      }
    }
  } catch (error) {
    triggerAlert("Szerver hiba mentés közben.", "error");
  } finally {
    setIsActionLoading(false);
  }
};
```

10.1.ábra: A napi napló összeállítását és backendre küldését végző aszinkron kliensoldali függvény.

A függvény először összeállítja a JSON adatcsomagot (payload), automatikusan hozzárendelve az aktuális dátumot és a bejelentkezett kliens azonosítóját. Ezt követően egy HTTP POST kéréssel továbbítja az adatokat a FastAPI backend /api/client/log végpontjára. Sikeres válasz esetén a frontend állapota (state) oldalfrissítés nélkül frissül: a kliens szériája (streak) újraszámolódik, a mérőszámok visszaállnak alapértékre, és egy értesítés jelzi a sikert.

10.2. Mesterséges Intelligencia elemzés

A rendszer egyik legkomplexebb funkciója a Google Gemini AI-alapú prediktív elemzése. A hagyományos adatlekérdezésekkel ellentétben itt a rendszer nyers biometrikus naplóból készít azonnal felhasználható, strukturált értékelést. A logika szerveroldalon, a FastAPI generate_client_analysis végpontjában valósul meg (10.2.1. ábra).

```
@app.post("/api/generate-client-analysis")
def generate_client_analysis(req: ClientAnalysisRequest):
    system_instruction = (
        "Te egy profi fitnesz adatelemző és prediktív AI vagy a Boosted platformon. "
        "A válaszod KIZÁRÓLAG egy érvényes JSON objektum lehet, markdown nélkül. "
        "A JSON KÖTELEZŐ struktúrája pontosan a következő legyen: "
        "{\n\"summary_text\": \"...\", \"risk_status\": \"Kritikus, Figyelmeztetés vagy Stabil\", \"risk_desc\": \"...\", \"\n\"risk_action\": \"...\", \"goal_status\": \"Kiváló, Stagnál vagy Visszaesés\", \"goal_metric\": \"Rövid metrika, pl. -2kg vagy 5 nap\", \"\n\"goal_desc\": \"...\", \"goal_action\": \"...\", \"pattern1_icon\": \"Egyetlen emoji\", \"pattern1_title\": \"...\", \"\n\"pattern1_desc\": \"...\", \"pattern2_icon\": \"Egyetlen emoji\", \"pattern2_title\": \"...\", \"pattern2_desc\": \"...\"}."
        "A szövegek magyar nyelvűek, lényegretörőek és professzionálisak legyenek."
    )

    try:
        model = genai.GenerativeModel(
            model_name="gemini-3.1-flash-lite-preview",
            system_instruction=system_instruction,
            generation_config={"response_mime_type": "application/json"}
        )
        response = model.generate_content(req.context_data)
        return json.loads(response.text)
    except Exception as e:
        print(f"Hiba a Kliens AI generálás során: {str(e)}")
        raise HTTPException(status_code=500, detail="Az AI motor nem tudta feldolgozni a Kliens adatokat.")
```

10.2.ábra: A Gemini mesterséges intelligencia API hívása.

A kódrészleten jól látható a Prompt Engineering alkalmazása. A system_instruction változóban a mesterséges intelligencia megkapja a szerepkörét és a szigorú kimeneti szabályokat. A kód legfontosabb része a response_mime_type: "application/json" beállítás, amely garantálja, hogy a modell szabad szöveg helyett egy szigorúan kulcs-érték párokból álló JSON objektumot adjon vissza. Ezt az adatot a szerver azonnal továbbítja a frontendnek, ami a dinamikus kártyákat rendereli.

10.3. Kapcsolat megszakítása és edzői értékelés

A platform lehetőséget biztosít az edző és a kliens közötti kapcsolat felbontására. Ezt az oldal alján található, biztonsági megerősítést kérő panelen indíthatják el a felhasználók. A folyamatot a handleDisconnectConfirm() aszinkron függvény vezérli (10.3.1. ábra), amely a bejelentkezett felhasználó szerepköre alapján dinamikusan határozza meg a kérés paramétereit.

```

const handleDisconnectConfirm = async () => {
  setIsActionLoading(true);
  try {
    // JAVÍTÁS: Itt helyben ellenőrizzük a szerepkört!
    const checkIsCoach = userRole === "COACH";

    const targetClientId = checkIsCoach ? selectedClient.id : userId;
    const targetCoachId = checkIsCoach ? coachId : (assignedCoachData?.id || coachId);

    const res = await fetch("https://boosted-backend-bvvv.onrender.com/api/disconnect", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ client_id: targetClientId, coach_id: targetCoachId })
    });

    if (res.ok) {
      setIsDisconnectModalOpen(false);
      if (checkIsCoach) {
        triggerAlert("Kapcsolat megszakítva a klienssel.", "success");
        setClients(clients.filter(c => c.id !== selectedClient.id));
        setSelectedClient(null);
      } else {
        // Ha a kliens bontott kapcsolatot, nyissuk meg az értékelő ablakot!
        setIsReviewModalOpen(true);
      }
    } else {
      triggerAlert("Hiba a kapcsolat megszakításakor.", "error");
    }
  } catch (err) {
    console.error(err); // Ha legközelebb hiba van, lássuk a böngésző konzoljában!
    triggerAlert("Rendszer hiba a megszakításakor.", "error");
  } finally {
    setIsActionLoading(false);
  }
};

```

10.3.ábra: A kapcsolat megszakítását és a folyamat utáni értékelő ablak megnyitását vezérlő logika.

A függvény a biztonsági ellenőrzések után meghívja a backend /api/disconnect végpontját, ami véglegesen törli a kliens fiókját és az adatait. Amennyiben kliens bontja fel a kapcsolatot, a React állapotkezelője (setIsReviewModalOpen(true)) automatikusan megnyitja a csillagos értékelő ablakot, amellyel a korábbi edző munkáját minősítheti a platformon.

11. Tesztelés

A szoftver minőségbiztosítása és a hibamentes működés garantálása érdekében az alkalmazás szerveroldali logikája automatizált teszteken esik át. A modern webalkalmazásoknál kritikus fontosságú, hogy a backend API végpontjai robusztusak legyenek, és ne csak a várt bemeneteket kezeljék helyesen, hanem a kivételes eseteket is. A teszteléshez a Python ökoszisztémában iparági standardnak számító pytest keretrendszert, valamint a FastAPI beépített TestClient modulját alkalmaztam, amely lehetővé teszi a HTTP kérések szimulálását a szerver tényleges elindítása nélkül.

11.1. Egységtesztek (Unit testing)

A fejlesztés során összesen **10 darab átfogó egységteszt** készült, amelyek a backend legkritikusabb végpontjait és védelmi vonalait vizsgálják. A tesztek kialakításánál kiemelt szempont volt, hogy ne csak a helyes, elvárt működést (Happy Path) ellenőrizzék, hanem a hibaágakat (Negative Testing) is.

- **Hitelesítés és JWT védelem tesztelése:** Külön tesztesetek vizsgálják a JWT tokenek generálását és az állapotmentes munkameneteket. A tesztek bizonyítják, hogy a rendszer sikeresen azonosítja és elutasítja a lejárt, manipulált vagy hiányzó tokeneket tartalmazó kéréseket, a szabványoknak megfelelő 401 Unauthorized HTTP státuszkódot visszaadva, ezzel védve a zárt végpontokat (pl. profil frissítése).
- **Bemeneti adatok validációja (Pydantic):** A Pydantic sémák (DTO-k) működését szándékosan hibás adatszerkezetek beküldésével teszteltem. Például a tesztek ellenőrzik, mi történik, ha az AI Chat végpontra nem küldünk üzenetlistát, vagy ha egy regisztrációs űrlapból hiányzik a jelszó. A szerver ezekben az esetekben biztonságosan, 422 Unprocessable Entity hibakóddal válaszol.
- **Üzleti logika és kivételkezelés:** A tesztek kiterjednek a komplex üzleti szabályok és a relációs hibák kezelésére is. Olyan folyamatokat vizsgálnak, mint például a "Boost" motivációs pontok küldése vagy a kapcsolat megszakítása nem létező kliensekkel (amelyeket a rendszer 404 Not Found hibával hárít el), illetve garantálják, hogy a heti edzéstervek kezdőnapja kizárólag hétfő lehessen (400 Bad Request).

Az automatizált tesztkészlet integrálása nagymértékben stabilizálta a projektet, mivel a kód refaktorálása és az új funkciók hozzáadása során azonnali, automatikus visszajelzést biztosított a rendszer magjának integritásáról.

12. Mesterséges intelligencia használata a fejlesztés során

A modern szoftvermérnöki gyakorlatnak napjainkra szerves részévé vált a mesterséges intelligencia (MI) alapú asszisztensek használata. A Boosted webalkalmazás fejlesztése során az MI nem a programozói munkát helyettesítő eszközként, hanem egyfajta "junior páros-programozóként" (pair-programmer) és hibakeresési (debugging) segédként volt jelen. Ennek a fejezetnek a célja, hogy transzparensen és kritikus mérnöki szemlélettel mutassa be a fejlesztés során alkalmazott nyelvi modellek használatát, azok előnyeit, valamint a velük járó veszélyeket és korlátokat.

12.1. Alkalmazott eszközök és feladatkörök

A projekt során a teljes rendszerarchitektúrát, az adatbázis-modellt és a funkcionális logikát magam terveztem meg és kódoltam le. Az ismétlődő, monoton feladatok felgyorsítására a GitHub Copilot szolgáltatását vettem igénybe a VS Code fejlesztőkörnyezeten belül. Ez az eszköz a már általam megírt kódminták alapján kiválóan alkalmas volt a statikus Tailwind CSS osztályok automatikus kiegészítésére, vagy a Pydantic sémák adattagjainak gyorsabb gépelésére.

A komplexebb hibakereséshez és a dokumentáció szakmai formázásához a Google Gemini (Gemini 3.1 Pro) webes felületét használtam technikai konzultánsként. A modell az alábbi feladatokban nyújtott támogatást:

1. **Szintaktikai segítség és refaktorálás:** Az üzleti logikát és a FastAPI végpontok alapját jómagam írtam, azonban a JWT (JSON Web Token) alapú autentikáció implementálásakor az iparági szabványoknak megfelelő szintaktika (boilerplate) gyors felépítésében, illetve az elavult SHA-256 algoritmus bcrypt titkosításra való cseréjének technikai kivitelezésében az MI nyújtott iránymutatást.
2. **Tesztek kiterjesztése:** A pytest alapú egységtesztek logikáját és a tesztelendő hibaágakat (pl. mi történik, ha egy heti terv nem hétfővel kezdődik) én határoztam meg. Miután megírtam az első teszteseteket, az MI-t arra használtam, hogy a megadott minta alapján gyorsabban generálja le a további 7-8 végpont szintaktikailag helyes tesztkódját.
3. **Hibakeresés (Debugging):** A kliensoldali és szerveroldali aszinkron kommunikáció során fellépő HTTP hibák (401, 422, 500) gyors diagnosztizálása a hálózati válaszok és a hibaüzenetek (stack trace) közös elemzésével.

12.2. Az MI kimenetének validációja és konkrét hibái

Mivel a rendszer magját én terveztem, alapszabályként alkalmaztam, hogy az MI által javasolt kódrészlet sosem tekinthető kész megoldásnak, csupán egy tesztelendő hipotézisnek. Minden javaslatot manuális kódolvasás (code review) és tesztelés követett. Ez a validáció elengedhetetlen volt, mert az MI többször is logikai hibákat vétett, vagy tévesen értelmezte a projekt struktúráját.

Egy konkrét eset volt például a kliens és az edző közötti "Kapcsolat megszakítása" funkció fejlesztése. A saját koncepcióm alapján megírt React kódban egy `ReferenceError` lépett fel. A Gemini által javasolt javítás meghívott egy `isCoach` nevű logikai változót az aszinkron függvényben. A javasolt kód szintaktikailag hibátlan volt, az MI azonban elvesztette a kontextust a nagy méretű fájlban: az `isCoach` változó nem a globális állapotterben (state), hanem csak lejjebb, a JSX renderelési blokkban volt definiálva. Ez a hiba a böngészőben "csendes" összeomlást okozott, amit az MI magától nem tudott azonosítani. A hibát a böngésző fejlesztői konzoljának (F12) manuális elemzésével és a hatókör (scope) lokális átrendezésével kellett megoldanom.

Egy másik tanulságos eset az adatbázis sémájának bővítéskor történt. Amikor az edzői értékeléshez manuálisan hozzáadtam az `average_rating` oszlopot a Python kódhoz, a tesztek futtatásakor a szerver `sqlite3.OperationalError: no such column` hibával leállt. Az MI a kód további refaktorálását javasolta, figyelmen kívül hagyva azt a tényt, hogy az alkalmazás nem használ adatbázis-migrációs eszközt (mint az Alembic), így maga az SQLite fájl nem frissült. A megoldást – a lokális adatbázisfájl törlését és a táblák újra-generálását – a korábbi fejlesztői tapasztalataimra támaszkodva, a modellt felülbírálvam kellett meglépnem.

12.3. Hol nem használtam mesterséges intelligenciát?

A projekt sikere szempontjából kritikus volt, hogy a szoftver gerincét alkotó területeken tudatosan mellőzzem az MI használatát.

- **Üzleti logika és rendszertervezés:** A platform alapvető koncepciója, a "Boost" gamifikációs pontrendszer szabályai, az adatbázis-táblák közötti relációk (egyed-kapcsolat diagram), valamint a kliens-edző jogosultsági körök megtervezése teljes mértékben saját mérnöki döntések eredménye.
- **UI/UX tervezés:** Bár a Tailwind CSS osztályok gépelésében a Copilot segített, a webalkalmazás képernyőterveit, a modern "Bento Grid" elrendezést és a felhasználói

útvonalakat (User Journey) vizuálisan, Figma szoftverben magam alkottam meg, bármiféle generatív képi MI bevonása nélkül.

- **Kritikus algoritmusok:** A szériaszámláló (streak) algoritmus logikája, amely dátumok alapján visszamenőlegesen ellenőrzi a naplózási folytonosságot, saját, kézzel írt implementáció.

12.4. Tanulságok és jövőkép

A folyamat legfőbb tanulsága, hogy a mesterséges intelligencia kiváló eszköz a prototipizálás és a repetitív kódolás akár 30-40%-os felgyorsítására, de nem pótolja a stabil szoftvermérnöki alaptudást. Komoly veszélyt jelent a "vak bizalom": amikor a projekt komplexitása megnőtt (például a page.js fájl mérete miatt), az MI egyre többször veszítette el a kontextust, és javasolt nem létező változókat vagy hibás hivatkozásokat.

A jövőbeli projekteknél az MI hatékonyabb használata érdekében még modulárisabb architektúrára törekszem. Kisebb, egyetlen felelősséggel rendelkező (Single Responsibility Principle) fájlok és komponensek használatával a nyelvi modellek kontextus-ablaka kevésbé terhelődik túl. Megtanultam továbbá, hogy az MI a hibakeresésben is csak akkor igazán hasznos, ha fejlesztőként képes vagyok pontosan értelmezni a rendszerarchitektúrát, és célzott adatokkal (stack trace, hálózati payload) irányítom a modellt a megoldás felé.

13. Tapasztalatok, továbbfejlesztési lehetőségek

A szakdolgozatom készítése előtt már tisztában voltam azzal, hogy a kliensoldalon React alapú technológiát fogok használni, mivel ennek a könyvtárnak a komponensalapú logikája áll hozzám a legközelebb. Ugyanakkor szerettem volna elmélyülni a modern Next.js keretrendszer nyújtotta lehetőségekben és a Tailwind CSS használatában. Szerveroldalon mindenképp Python alapú megoldást kerestem, hiszen a mesterséges intelligencia integrálása és az adatfeldolgozás Python környezetben a leghatékonyabb. Így esett a választás a rendkívül gyors és modern FastAPI keretrendszerre.

A tervezés előtt alapos kutatást végeztem a megfelelő vizualizációs könyvtárak (pl. Recharts) és a PDF generálás lehetőségei (jsPDF) terén. Miután megbizonyosodtam róla, hogy ezek a technológiák zökkenőmentesen együttműködnek a Reacttel, valamint a Google Gemini API is képes strukturált formátumban válaszolni, nekiálltam a tervezésnek. Létrehoztam a GitHub repozitóriumot, hogy a verziókezelés az első lépésektől biztosított legyen. A GitHub Commit funkcióját aktívan használtam a feladatok követésére: itt rögzítettem az ötleteket, az új funkciókat (mint az utólag bekerült kapcsolatbontás és értékelő rendszer), valamint a konzulensi visszajelzések alapján szükséges technikai javításokat is.

Az első lépések egyike a képernyőtervek és a koncepció megalkotása volt. Mivel a rendszer két teljesen eltérő felhasználói bázist (edzők és kliensek) szolgál ki, a reszponzivitás és a vizuális hierarchia kritikus volt. Kialakítottam egy modern, "Bento Grid" elrendezést és alkalmaztam a "Glassmorphism" stílusjegyeket. Külön kihívást, de egyben izgalmas feladatot jelentett a szerepkörök szerinti dinamikus színekódolás (lila/kék az edzőknek, smaragdzöld/narancs a klienseknek) implementálása, valamint a letisztult, csúszkákra épülő adatbeviteli űrlapok megtervezése.

A UI kialakítása után az adatbázis-architektúrával foglalkoztam. Megterveztem az egyed-kapcsolat diagramot, majd SQLite és az SQLAlchemy ORM segítségével létrehoztam a relációs táblákat. Menet közben nagy hangsúlyt fektettem a biztonságra: a fejlesztés egy pontján a korábbi autentikációs logikát egy sokkal robusztusabb, bcrypt titkosításon és JWT (JSON Web Token) tokeneken alapuló állapotmentes hitelesítési rendszerre cseréltem le, ami rendkívül hasznos tanulási folyamat volt számomra.

Ezután következett a kódolás legintenzívebb szakasza, amit nagyon élveztem. Kisebb nehézségek adódtak a kliens-szerver aszinkron kommunikáció, az állapotkezelés, és a Pydantic sémák szigorú adatvalidációjának összehangolásakor, de ezeket a tesztelések során gyorsan

azonosítottam és javítottam. A legizgalmasabb mérföldkő egyértelműen a Google Gemini AI bekötése és a megfelelő Prompt Engineering (rendszerutasítások) kidolgozása volt, elszigetelve a hibákat annak érdekében, hogy az AI a nyers biometrikus naplókából valóban strukturált JSON válaszokat és értékelhető predikciókat adjon.

Bár az alkalmazás jelenlegi formájában is egy kerek, működőképes egész, számos továbbfejlesztési lehetőséget tartogat. A legfontosabb a külső okoseszközök (pl. Apple Health, Garmin) API-jának integrálása lenne, így a klienseknek nem kellene manuálisan rögzíteniük az alvás- és lépésadatokat, azok automatikusan szinkronizálódnának a rendszerrel. Emellett egy dedikált mobilapplikáció (React Native alapokon) tovább növelné a felhasználói élményt a push értesítések révén. Szintén hasznos funkció lenne a táplálkozás részletesebb, makrotápanyag szintű követése, összekötve egy okos ételadatbázissal.

A webalkalmazás élesítése (deployment) során sikeresen implementáltam egy szétválasztott, modern felhőarchitektúrát (Vercel, Render, Neon.tech PostgreSQL), integrálva a GitHub alapú folyamatos telepítést (CI/CD). A rendszer jelenlegi iterációja (MVP) a gyors tartalomcserére fókuszálva állapotvezérelt (state-based) Single Page Application (SPA) architektúrát használ. A jövőbeli technikai fejlesztések egyik fő iránya a Next.js natív, fájlrendszer-alapú útválasztására (App Router) való teljes átállás. Ez biztosítaná a dedikált URL-ek használatát, ezáltal teljesskörűen támogatva a böngészők natív navigációs (előre-hátra és frissítés) funkcióit, valamint tovább javítaná a rendszer keresőoptimalizálását (SEO).

A monetizáció és a szolgáltatáskínálat bővítése jegyében a jövőben tervezem a platformba integrált PRO csomag valós, előfizetéses funkciókkal történő feltöltését. Ez olyan exkluzív mesterséges intelligencia lehetőségeket nyitna meg, mint az edzők számára az AI által támogatott automatikus edzésterv-optimalizálás, a klienseknek pedig egy kontextus-tudatos, személyre szabott chat-asszisztens. A közösségépítés és a szakmai versenyszellem ösztönzése érdekében egy dinamikus edzői ranglista (leaderboard) kialakítása is a tervek között szerepel. Ez a beérkezett ügyfélértékelések és az aktivitás alapján rangsorolná a szakértőket, lehetővé téve a városonkénti vagy akár országos szintű szűrést, ami komoly marketingértéket jelentene a platformon dolgozó edzők számára. Ezzel párhuzamosan a kliensi oldalon a jelenlegi "széria" és motivációs rendszert kiterjesztett gamifikációs elemekkel – például megszerezhető jelvényekkel (badge), mérföldkövekkel és szintlépésekkel – bővíteném a hosszú távú elköteleződés és a felhasználói élmény maximalizálása érdekében.

Összességében úgy gondolom, hogy a Boosted platformmal sikerült egy olyan modern, mesterséges intelligenciával támogatott ökoszisztémát létrehoznom, amely funkcióinak köszönhetően könnyen és gyorsan használható. Valóban képes proaktív hidat képezni az edzők és klienseik között, optimalizálva a közös munkát és minimalizálva a kiégés vagy a sérülések kockázatát.

Irodalomjegyzék

1. bcrypt: <https://pypi.org/project/bcrypt/> (Utolsó megtekintés: 2026. 04. 29.)
2. FastAPI: <https://fastapi.tiangolo.com/> (Utolsó megtekintés: 2026. 04. 29.)
3. Figma: <https://www.figma.com/> (Utolsó megtekintés: 2026. 04. 28.)
4. GitHub: <https://github.com/> (Utolsó megtekintés: 2026. 04. 29.)
5. Google Gemini AI: <https://ai.google.dev/> (Utolsó megtekintés: 2026. 04. 29.)
6. jsPDF: <https://github.com/parallax/jsPDF> (Utolsó megtekintés: 2026. 04. 29.)
7. MyFitnessPal: <https://www.myfitnesspal.com/> (Utolsó megtekintés: 2026. 04. 28.)
8. Neon.tech: <https://neon.tech/> (Utolsó megtekintés: 2026.04.30.)
9. Next.js: <https://nextjs.org/> (Utolsó megtekintés: 2026. 04. 29.)
10. Pydantic: <https://docs.pydantic.dev/> (Utolsó megtekintés: 2026. 04. 29.)
11. Python: <https://www.python.org/> (Utolsó megtekintés: 2026. 04. 29.)
12. React: <https://react.dev/> (Utolsó megtekintés: 2026. 04. 29.)
13. Recharts: <https://recharts.org/> (Utolsó megtekintés: 2026. 04. 29.)
14. Render: <https://render.com/> (Utolsó megtekintés: 2026.04.30.)
15. SQLAlchemy: <https://www.sqlalchemy.org/> (Utolsó megtekintés: 2026. 04. 29.)
16. SQLite: <https://www.sqlite.org/> (Utolsó megtekintés: 2026. 04. 29.)
17. Tailwind CSS: <https://tailwindcss.com/> (Utolsó megtekintés: 2026. 04. 29.)
18. Trainerize: <https://www.trainerize.com/> (Utolsó megtekintés: 2026. 04. 28.)
19. TrainingPeaks: <https://www.trainingpeaks.com/> (Utolsó megtekintés: 2026. 04. 28.)
20. Uvicorn: <https://www.uvicorn.org/> (Utolsó megtekintés: 2026. 04. 29.)
21. Vercel: <https://vercel.com/> (Utolsó megtekintés: 2026.04.30.)
22. Visual Studio Code: <https://code.visualstudio.com/> (Utolsó megtekintés: 2026. 04. 29.)

Nyilatkozat

Alulírott Földes Balázs programtervező informatikus szakos hallgató, kijelentem, hogy a dolgozatomat a Szegedi Tudományegyetem, Informatikai Intézet Szoftverfejlesztés Tanszékén készítettem, programtervező informatikus BSc diploma megszerzése érdekében.

Kijelentem, hogy a dolgozatot más szakon korábban nem védtem meg, saját munkám eredménye, és csak a hivatkozott forrásokat (szakirodalom, eszközök, stb.) használtam fel. Tudomásul veszem, hogy a szakdolgozatomat / diplomamunkámat a Szegedi Tudományegyetem Diplomamunka Repozitóriumban tárolja.

Szeged, 2026.05.05.

Aláírás

Köszönetnyilvánítás

Szeretnék köszönetet mondani elsősorban a témavezetőmnek, Dr. Bilicki Vilmos egyetemi adjunktusnak, amiért segítette és felügyelte a munkámat.

Emellett szeretnék köszönetet mondani a családomnak, akik teljes szívükkel támogattak nemcsak a szakdolgozatom elkészítése közben, hanem az egyetemi éveim alatt is. Valamint köszönöm a megértésüket, amikor fel kellett áldoznom a velük eltöltött időt a szakdolgozatom haladása érdekében és köszönöm a segítségüket, tanácsaikat.

Valamint nem utolsó sorban szeretnék köszönetet mondani a barátnőmnek is. A nehéz időszakokban mindig a rendelkezésére álltam és türelmesen meghallgatta az összes panaszom, ha úgy volt akkor segített azokon túllendülni és tanácsokkal látott el.

Elektronikus mellékletek

1. A webalkalmazás GitHub elérhetősége: <https://github.com/BalazsFoldes/Boosted-2.0> (Utolsó megtekintés: 2026.04.29.)
2. A webalkalmazás hosztolt linkje: <https://boosted-app.vercel.app/> (Utolsó megtekintés: 2026.04.30.)